

TÉCNICAS DIGITALES II

COMPENDIO TEÓRICO COMPLETO PARA EL 1ER PARCIAL

Guía de estudio exhaustiva, rigurosa y práctica que cubre la totalidad de los contenidos evaluados:
Arquitectura de Computadores, Métricas de Rendimiento, Núcleo ARM Cortex-M3 (ARMv7-M), Toolchain (GCC/LD/Startup), NVIC y Excepciones, GPIO/RCC, Protocolos Serie (SPI, I2C, UART) y Kernel FreeRTOS (Tareas, Colas, Interrupciones y Procesamiento Diferido).

Cátedra: Dr. Ing. Steiner Guillermo

Estudiante: Gastón

Materia: Técnicas Digitales II (4to Año)

Hardware de Referencia: STM32F103C8T6 ("Blue Pill", ARM Cortex-M3 @ 72 MHz)

Software & Toolchain: GNU GCC (arm-none-eabi), FreeRTOS Kernel v10.x, CMSIS Core

ÍNDICE GENERAL TEMÁTICO

MÓDULO 1: ARQUITECTURA DE COMPUTADORES Y RENDIMIENTO

1. Introducción a los Microprocesadores, Clasificación y las 8 Grandes Ideas
2. Medición y Ecuación del Rendimiento, Factores de Impacto y el Muro de Potencia

MÓDULO 2: ARQUITECTURA ARM CORTEX-M Y MAPEO DE MEMORIA

3. Arquitectura ARMv7-M, RISC vs CISC y el Núcleo Cortex-M3
4. Mapeo de Memoria, Decodificador de Direcciones y Clasificación de Periféricos

MÓDULO 3: TOOLCHAIN, ENLAZADO Y ARRANQUE (BARE-METAL)

5. Flujo de Compilación: Compilador, Ensamblador (2 Pasadas) y Archivo Objeto (.o)
6. Enlazado (Linker), Linker Scripts (.ld), Dualidad LMA vs VMA y Código de Startup (crt.s)

MÓDULO 4: CONTROLADOR DE INTERRUPCIONES Y EXCEPCIONES (NVIC)

7. Modelo de Excepciones: Clasificación, Datapath y Vectorización
8. NVIC en Detalle: Prioridades de Grupo vs Sub-prioridad, Reubicación VTOR y Registros SCB
9. Mecanismo de Auto-Stacking por Hardware, EXC_RETURN y Retorno de Excepción

MÓDULO 5: PERIFÉRICOS Y PROTOCOLOS DE COMUNICACIÓN SERIE

10. Capas Jerárquicas de Activación de Periféricos y Configuración de GPIO/RCC
11. Protocolo SPI: Topologías, Modos de Reloj (CPOL/CPHA), Registros y Código en C
12. Protocolo I2C: Open-Drain, Start/Stop, ACK/NACK, Arbitraje, 10-bits y Modo High-Speed
13. Protocolo UART / USART: RS-232, Trama, Paridad, Sobremuestreo, Cálculo de BRR y Handshaking

MÓDULO 6: SISTEMAS OPERATIVOS EN TIEMPO REAL (FreeRTOS)

14. Paradigma Super-Loop vs RTOS, Requisitos Hard/Soft Real-Time y Ventajas
15. Gestión de Tareas: Anatomía, Estados, Scheduling, Inanición y Delays (vTaskDelay vs vTaskDelayUntil)
16. Tarea Idle, Idle Hook, Prioridades Dinámicas y Ciclo de Vida
17. Colas de Mensajes (Queues): Copia por Valor, Bloqueo, Múltiples Emisores y Punteros
18. Gestión de Interrupciones: API FromISR, pxHigherPriorityTaskWoken y Procesamiento Diferido con Semáforos Binarios

MÓDULO 7: BANCO DE PREGUNTAS TÍPICAS DE EXAMEN Y RESPUESTAS MODELO (30 Preguntas)

MÓDULO 1: ARQUITECTURA DE COMPUTADORES Y RENDIMIENTO

Capítulo 1: Introducción a los Microprocesadores y Clasificación

1.1. La Revolución de la Información y la Ley de Moore

A lo largo de la historia humana, las revoluciones agrícola e industrial transformaron la capacidad física del hombre. En las últimas décadas, la acelerada evolución del hardware digital ha dado lugar a la **Revolución de la Información**: un cambio de paradigma en el cual la computación redefine la ciencia, la industria, las comunicaciones y la sociedad moderna.

El motor tecnológico de este crecimiento exponencial fue formulado por **Gordon Moore en 1965**:

Enunciado de la Ley de Moore:

La densidad de transistores integrados en un circuito integrado se duplica aproximadamente cada 18 a 24 meses.

Consecuencias directas de la Ley de Moore:

- **Reducción drástica del costo por transistor:** Hace viable económicamente fabricar dispositivos de consumo masivo con miles de millones de transistores.
- **Aumento de la velocidad de conmutación:** Al reducirse las dimensiones físicas de los transistores (canal más corto y menor capacitancia parasitaria), estos conmutan más rápido, permitiendo mayores frecuencias de reloj.
- **Impacto en aplicaciones cotidianas:** Electrónica automotriz (control de inyección, seguridad activa ABS y airbags), telefonía celular global, secuenciación del genoma humano a bajo costo, World Wide Web y motores de búsqueda a escala planetaria.

1.2. Clasificación de la Computación Moderna

Categoría	Objetivo Primario	Características y Criterios de Diseño	Ejemplos Típicos
Computadores Personales (PC)	Optimización Precio / Rendimiento para un único usuario	Ejecución de software de terceros diverso, interfaces gráficas interactivas, balance de costo razonable.	Laptops, computadoras de escritorio, estaciones de trabajo.
Servidores y Cloud Computing	Rendimiento agregado (Throughput) y Alta Disponibilidad	Orientados a cargas masivas de trabajo o millones de peticiones concurrentes pequeñas. Máxima confiabilidad, tolerancia a fallos 24/7 y escalabilidad.	Servidores web, bases de datos empresariales, supercomputadores, WSC (Warehouse Scale Computers).
Sistemas Embebidos (Embedded Systems)	Ejecución confiable y dedicada de una función específica	Es la clase de computadoras más grande en volumen de la industria. Integrados dentro de un producto mayor. El software es considerado parte fija e integral del hardware.	ECUs de automóviles, electrodomésticos, instrumental médico, controladores industriales, IoT.

1.3. Restricciones Críticas de Diseño en Sistemas Embebidos

Los sistemas embebidos presentan las condiciones de contorno más exigentes de la ingeniería electrónica:

1. **Restricciones de Costo Extremas:** Se fabrican en series de cientos de miles o millones de unidades. Una diferencia de pocos centavos de dólar en el microcontrolador o sus componentes pasivos define la viabilidad comercial del producto.
2. **Consumo de Energía:** Crítico en dispositivos a batería (sensores remotos, wearables) y en equipos sin ventiladores (disipación exclusivamente pasiva para evitar acumulación de polvo y fallas mecánicas).
3. **Confiabilidad y Disponibilidad (Baja tolerancia a fallas):** Mientras que en un producto de consumo (como un Smart TV) una falla implica el descontento del cliente, en sistemas críticos (frenos ABS, control de turbinas, marcapasos, aviónica) un fallo del software o hardware implica **riesgo directo de vida**.
4. **Interacción con el Mundo Real en Tiempo Real (Real-Time):** Deben procesar señales analógicas/digitales y responder a estímulos externos dentro de intervalos de tiempo estrictos y garantizados (determinismo temporal).

1.4. Términos y Unidades de Almacenamiento (Estándar Decimal vs Binario)

Históricamente existió ambigüedad entre las unidades de base 10 (usadas por fabricantes de discos y el Sistema Internacional SI) y las unidades de base 2 (usadas por desarrolladores de software y arquitectura de memoria). En 1998, la norma **IEC 80000-13** formalizó los

prefijos binarios:

Término Decimal (SI)	Abrev.	Valor Decimal	Término Binario (IEC)	Abrev.	Valor Binario	Discrepancia (% Mayor)
Kilobyte	KB	10^3 (1.000)	Kibibyte	KiB	2^{10} (1.024)	+2,4 %
Megabyte	MB	10^6 (1.000.000)	Mebibyte	MiB	2^{20} (1.048.576)	+4,9 %
Gigabyte	GB	10^9 (10^9)	Gibibyte	GiB	2^{30} (1.073.741.824)	+7,4 %
Terabyte	TB	10^{12}	Tebibyte	TiB	2^{40} (1.099.511.627.776)	+10,0 %
Petabyte	PB	10^{15}	Pebibyte	PiB	2^{50}	+12,6 %
Exabyte	EB	10^{18}	Exbibyte	EiB	2^{60}	+15,3 %
Zettabyte	ZB	10^{21}	Zebibyte	ZiB	2^{70}	+18,0 %
Yottabyte	YB	10^{24}	Yobibyte	YiB	2^{80}	+20,9 %

OJO - Trampa de Examen:

En memoria principal RAM y direccionamiento interno de microcontroladores se opera en múltiplos de 2 (KiB = 1024 bytes, MiB = 1024 KiB). Confundir 64 KB con 64 KiB en cálculos de direccionamiento puede provocar errores de límites en tablas de vectores y Linker Scripts.

1.5. La Era Post-PC, PMD y Warehouse Scale Computers

- **PMD (Personal Mobile Devices):** Teléfonos inteligentes y tabletas que reemplazaron a la PC como dispositivo primario. Requisitos: ultra bajo consumo, interfaces gestuales/táctiles y conectividad inalámbrica continua.
- **WSC (Warehouse Scale Computers):** Centros de datos masivos (Google, Amazon AWS, Microsoft Azure) diseñados como una sola computadora gigantesca de escala de almacén para procesar cargas masivas.
- **SaaS (Software as a Service):** Aplicaciones cuyo procesamiento central y almacenamiento residen en la nube y no en el dispositivo local (Google Drive, Microsoft 365, Zoom).

1.6. Las 8 Grandes Ideas de la Arquitectura de Computadores

David Patterson y John Hennessy resumen 60 años de evolución arquitectónica en 8 principios fundamentales:

1. **Diseño para la Ley de Moore:** Debido a que el diseño de un microprocesador toma entre 2 y 4 años, los arquitectos deben diseñar proyectando la tecnología y densidad que existirá al finalizar el proyecto, no la actual.
2. **Uso de la Abstracción para Simplificar el Diseño:** Ocultar detalles de bajo nivel mediante capas jerárquicas para que el diseñador de niveles superiores no sea abrumado por la complejidad del silicio.
3. **Hacer el Caso Común más Rápido (Make the Common Case Fast):** Optimizar exhaustivamente las instrucciones y trayectorias de datos que se ejecutan el 90% del tiempo (Ley de Amdahl), en lugar de optimizar casos raros.
4. **Rendimiento vía Paralelismo:** Mejorar la velocidad ejecutando múltiples operaciones simultáneamente (múltiples ALUs, procesadores multinúcleo).
5. **Rendimiento vía Pipelining (Segmentación del Cauce):** Superponer las etapas de ejecución de instrucciones (Fetch, Decode, Execute, Memory, Writeback) como una línea de ensamblaje en una fábrica.
6. **Rendimiento vía Predicción:** Anticipar el resultado de una operación o bifurcación condicional antes de que se complete, asumiendo que es más rápido corregir un error eventual que esperar ociosos.
7. **Jerarquía de Memorias:** Estructurar el almacenamiento en niveles: memorias rápidas, pequeñas y caras (SRAM Caché) cerca de la CPU; y memorias lentas, grandes y baratas (DRAM, Flash) en niveles inferiores.
8. **Dependencia y Confiabilidad vía Redundancia:** Incorporar hardware de respaldo (servidores duplicados, bits de paridad, ECC) para detectar fallos y continuar operando sin interrupción.

1.7. Capas de Abstracción de Software y Flujo de Traducción

El software se estructura en capas de abstracción para independizar el desarrollo de aplicaciones de los detalles físicos del microcontrolador:

```

+-----+
| Software de Aplicación (Procesamiento, Lógica C) |
+-----+
| Software de Sistema: Sistema Operativo (RTOS) + |
| Compilador (GNU GCC) + Bibliotecas (CMSIS/HAL) |
+-----+
| Hardware Digital (CPU, Registros, Buses, Periféricos)|
+-----+

```

Cadena de Transformación del Código al Hardware:

1. **Lenguaje de Alto Nivel (C, C++):** Expresa algoritmos de forma estructurada e independiente del procesador.
2. **Compilador:** Traduce el código C a código Ensamblador específico de la arquitectura.
3. **Lenguaje Ensamblador:** Representación simbólica legible por humanos de las instrucciones de máquina mediante mnemónicos (ej. `ADD R0, R0, R1`).
4. **Ensamblador:** Convierte los mnemónicos en código binario puro (Código Máquina: ceros y unos).
5. **Código Máquina:** Secuencia binaria ejecutada directamente por los circuitos lógicos del procesador.

Pregunta Clave de Examen: ¿Por qué es vital el Compilador en esta jerarquía?

1.

Manejo de la Complejidad:

Evita la tarea inviable de programar manualmente millones de transistores en binario.

2.

Independencia del Hardware (Portabilidad):

Permite que el mismo algoritmo escrito en C se ejecute en una PC (x86), en un servidor (ARMv8) o en un microcontrolador (Cortex-M3), delegando en el compilador la traducción al dialecto de máquina de cada arquitectura física.

1.8. Los 5 Componentes Fundamentales del Hardware

Cualquier sistema de cómputo se compone de cinco bloques básicos:

1. **Entrada (Input):** Mecanismos de recepción de datos del entorno exterior (sensores, pines GPIO en entrada, teclados, transceptores UART/SPI).
2. **Salida (Output):** Mecanismos de emisión hacia el exterior (pantallas, actuadores, pines GPIO en salida, relés).
3. **Memoria (Memory):** Almacén de programas ejecutables (Flash) y datos temporales/variables (RAM).
4. **Ruta de Datos (Datapath):** Conjunto de registros internos y Unidad Aritmético-Lógica (ALU) que ejecuta los cálculos matemáticos y lógicos.
5. **Unidad de Control (Control):** Circuito secuencial que decodifica las instrucciones y comanda al Datapath, a la memoria y a los periféricos.

Definición: La **CPU (Unidad Central de Procesamiento)** está conformada por la unión de la **Ruta de Datos (Datapath) + Unidad de Control**.

1.9. Jerarquía de Memorias

Existe una enorme brecha de velocidad entre la CPU y la memoria principal (DRAM): mientras la CPU opera a frecuencias de gigahercios, la DRAM externa requiere decenas de ciclos de reloj para responder. La solución es la jerarquía de memorias:

- **Memoria Caché (SRAM - Static RAM):** Implementada con celdas de 6 transistores por bit. No necesita refresco. Tiempos de acceso ultrarrápidos (1 a 2 ciclos), pero baja densidad y alto costo por bit. Ubicada dentro del silicio del procesador.
- **Memoria Principal (DRAM - Dynamic RAM):** Implementada con 1 transistor y 1 capacitor por bit. Muy alta densidad y bajo costo, pero mayor latencia debido a la necesidad periódica de ciclos de refresco para no perder la carga del capacitor.
- **Almacenamiento Secundario No Volátil:** Retiene la información sin suministro eléctrico.
 - **Memoria Flash:** Rápida, de estado sólido, sin partes móviles. Utilizada para guardar el código en microcontroladores y en discos SSD.
 - **Discos Magnéticos / Ópticos:** Masivos, mecánicos, mayor latencia de acceso.

1.10. La Interfaz Hardware/Software: ISA y ABI

- **ISA (Instruction Set Architecture):** Es la interfaz abstracta formal entre el hardware y el software de bajo nivel. Engloba el conjunto de instrucciones máquina, el modelo y número de registros, los modos de direccionamiento de memoria y el modelo de excepciones. Permite la **independencia de implementación**: chips con diferente diseño circuital interno (ej. Intel Core i7 y AMD Ryzen) pueden ejecutar exactamente el mismo software porque comparten la misma ISA (x86-64).
- **ABI (Application Binary Interface):** Es la especificación que complementa a la ISA definiendo convenciones a nivel binario: cómo se pasan argumentos a funciones (en qué registros o en la pila), cómo se retorna un valor, alineación de estructuras de datos en memoria y cómo se invocan los servicios del Sistema Operativo.
- **EABI (Embedded ABI):** Estándar de ABI optimizado para sistemas embebidos, diseñado para minimizar el consumo de memoria RAM y el código generado.

1.11. Familias de Arquitecturas Principales

Arquitectura	Filosofía	Modelo de Negocio / Características Clave	Aplicación Principal
ARM (Advanced RISC Machines)	RISC	ARM no fabrica chips; vende licencias de núcleos de silicio a terceros (ST, NXP, Apple, TI). Máxima eficiencia energética. Se divide en Cortex-A (aplicaciones/smartphones), Cortex-R (tiempo real crítico automotriz) y Cortex-M (microcontroladores).	98% de smartphones mundiales, IoT, sistemas embebidos, servidores cloud eficientes (Graviton).
x86 / x64 (Intel & AMD)	CISC	Conjunto de instrucciones complejo y variable. Fuerte énfasis en la retrocompatibilidad de 40 años. Las implementaciones modernas decodifican internamente las instrucciones CISC en micro-operaciones tipo RISC.	Computadoras personales, estaciones de trabajo y servidores de cómputo pesado.
RISC-V	RISC	ISA abierta, libre y sin pago de regalías (Open-Source), nacida en UC Berkeley. Arquitectura modular: núcleo base minimalista más extensiones opcionales (FPU, vectorial, compresión).	Microcontroladores modernos, aceleradores de Inteligencia Artificial, investigación académica.
MIPS	RISC	La expresión más pura de la filosofía RISC académica. Instrucciones de longitud fija y segmentación limpia de 5 etapas.	Routers de red, módems, consolas históricas (PS1, N64), enseñanza de arquitectura.
AVR (Microchip / Atmel)	RISC (8 bits)	Arquitectura tipo Harvard de 8 bits con 32 registros de trabajo, diseñada para ejecutar C de manera eficiente en chips económicos y de pocos pines.	Placas Arduino Uno, dispositivos domésticos de bajo costo.

Capítulo 2: Medición y Ecuación del Rendimiento, Factores de Impacto y el Muro de Potencia

2.1. Definición Formal de Rendimiento (Performance)

El rendimiento no es un concepto absoluto; su definición depende del tipo de sistema:

- **Computadora Personal (PC):** Mejor rendimiento significa menor **Tiempo de Respuesta (o Tiempo de Ejecución)**: tiempo transcurrido para completar una tarea individual.
- **Servidores y Centros de Datos:** Mejor rendimiento significa mayor **Ancho de Banda (Throughput o Tasa de Transferencia)**: cantidad de tareas procesadas por unidad de tiempo.

$$Rendimiento_X = \frac{1}{\text{Tiempo de Ejecucion}_X}$$

Si una computadora X es n veces más rápida que una computadora Y :

$$n = \frac{Rendimiento_X}{Rendimiento_Y} = \frac{\text{Tiempo de Ejecucion}_Y}{\text{Tiempo de Ejecucion}_X}$$

Regla de Evaluación:

Mejorar la CPU (mayor frecuencia o menor CPI) mejora tanto el tiempo de respuesta como el ancho de banda. En cambio, *agregar más procesadores en paralelo* únicamente mejora el ancho de banda, pero no reduce el tiempo de ejecución de un proceso monohilo secuencial.

2.2. Tiempo Transcurrido vs Tiempo de CPU

- **Tiempo Transcurrido (Wall-clock time o Elapsed time):** Tiempo total medido por un reloj de pared desde el inicio hasta el fin de la tarea. Incluye accesos a disco/flash, esperas por periféricos de E/S y el tiempo que el sistema operativo dedica a otros programas (multitarea).
- **Tiempo de CPU (CPU Execution Time):** Tiempo que el procesador dedica *exclusivamente* a ejecutar las instrucciones de nuestro programa. No incluye demoras por I/O ni procesos ajenos. Se desglosa en:
 - **Tiempo de CPU de Usuario (User CPU Time):** Tiempo ejecutando el código propio del programa.
 - **Tiempo de CPU de Sistema (System CPU Time):** Tiempo que el sistema operativo invierte en atender llamadas al sistema (syscalls) solicitadas por el programa.

2.3. Parámetros de Reloj y Ecuación Fundamental del Rendimiento

Los circuitos digitales sincrónicos operan regidos por una señal de reloj de frecuencia constante:

- **Período de Reloj (T_{clock}):** Duración temporal de un ciclo (tick), medido en segundos (o ns, ps).
- **Frecuencia de Reloj (f_{clock}):** Inverso del período, medido en Hertz (MHz, GHz): $f_{\text{clock}} = \frac{1}{T_{\text{clock}}}$.

La Ecuación del Rendimiento de CPU:

$$\text{Tiempo de CPU} = I \times \text{CPI} \times T_{\text{clock}} = \frac{I \times \text{CPI}}{f_{\text{clock}}}$$

Donde:

- **I (Instruction Count):** Número total de instrucciones dinámicas ejecutadas por el programa.
- **CPI (Cycles Per Instruction):** Promedio de ciclos de reloj necesarios para ejecutar una instrucción.

2.4. Cálculo del CPI Ponderado y Comparación de Código

En un procesador real, distintas clases de instrucciones requieren diferente cantidad de ciclos (ej. sumas en 1 ciclo, multiplicaciones en 3 ciclos, accesos a memoria en 4 ciclos). El CPI global promedio se calcula en base a la mezcla de instrucciones:

$$\text{CPI}_{\text{promedio}} = \frac{\sum_{i=1}^n (I_i \times \text{CPI}_i)}{\sum_{i=1}^n I_i} = \frac{\text{Total de Ciclos de Reloj}}{\text{Total de Instrucciones}}$$

Ejemplo Típico de Examen Resuelto:

Se dispone de una CPU con 3 clases de instrucciones: Clase A (CPI = 1), Clase B (CPI = 2) y Clase C (CPI = 3). Se evalúan dos secuencias de código generadas por compiladores distintos:

- **Secuencia 1:**

2 de tipo A, 1 de tipo B, 2 de tipo C. Total: 5 instrucciones.

- **Secuencia 2:**

4 de tipo A, 1 de tipo B, 1 de tipo C. Total: 6 instrucciones.

Cálculo:

- Ciclos Código 1 = $(2 \times 1) + (1 \times 2) + (2 \times 3) = 2 + 2 + 6 = 10$ ciclos.
 $CPI_1 = \frac{10}{5} = 2.0$.
- Ciclos Código 2 = $(4 \times 1) + (1 \times 2) + (1 \times 3) = 4 + 2 + 3 = 9$ ciclos.
 $CPI_2 = \frac{9}{6} = 1.5$.

Conclusión:

El Código 2 es más rápido (toma 9 ciclos frente a 10 ciclos del Código 1), a pesar de tener *mayor cantidad de instrucciones* ($6 > 5$). Esto demuestra que un programa con más instrucciones puede ser más veloz si su CPI promedio es significativamente menor.

2.5. Factores que Afectan las Variables del Rendimiento

Componente del Sistema	Afecta a	Mecanismo / Forma de Impacto
Algoritmo	I , posiblemente CPI	Determina el número total de operaciones lógicas requeridas y la eficiencia del flujo de control.
Lenguaje de Programación	I , CPI	Lenguajes de alto nivel con abstracciones complejas generan más instrucciones y estructuras con mayor CPI que lenguajes cercanos al hardware como C.
Compilador	I , CPI	La optimización del compilador (desenrollado de bucles, asignación de registros, reordenamiento de instrucciones) reduce tanto I como el CPI.
Arquitectura del Conjunto de Instrucciones (ISA)	I , CPI, f_{clock}	Una ISA CISC reduce I pero incrementa el CPI y limita la frecuencia. Una ISA RISC incrementa I pero reduce el CPI y permite frecuencias de reloj más altas.

2.6. El Muro de Potencia (Power Wall) y el Giro hacia Multicore

Desde 1970 hasta 2004, el rendimiento de los procesadores creció aumentando la frecuencia de reloj y reduciendo el voltaje de alimentación. En tecnología CMOS, la potencia disipada por conmutación (potencia dinámica) responde a la ecuación:

$$P_{\text{din}} \approx \frac{1}{2} \cdot C \cdot V^2 \cdot f$$

Donde C es la capacitancia de carga de los transistores, V es el voltaje de alimentación y f es la frecuencia de conmutación.

Durante tres décadas, al escalar la litografía se redujo la tensión de 5V a 1V. Esto permitió que la frecuencia aumentara 1000× mientras que la potencia solo aumentó 30×.

El Quiebre del Muro de Potencia (2004):

El escalado colapsó debido a dos límites físicos insalvables:

1. **Límite de Voltaje (Corrientes de Fuga / Leakage):**

Por debajo de ~0.8V a 1V, los transistores no pueden apagarse completamente; la corriente de fuga estática crece exponencialmente disipando calor incluso sin conmutar.

2. **Límite Térmico de Disipación:**

La densidad de calor (Watts/cm²) alcanzó niveles comparables a la tobera de un reactor nuclear, volviendo inviable la refrigeración por aire convencional económica.

El Cambio de Paradigma (2006 en adelante): Al no poder aumentar la frecuencia monohilo, la industria migró hacia procesadores **Multinúcleo (Multicore)** y multiprocesamiento. Para aprovechar el hardware actual, el software ya no se beneficia pasivamente del avance tecnológico; debe ser diseñado explícitamente como **software concurrente y distribuido en tareas/hilos**.

MÓDULO 2: ARQUITECTURA ARM CORTEX-M Y MAPEO DE MEMORIA

Capítulo 3: La Familia ARM y el Perfil Cortex-M3

3.1. Arquitectura vs Implementación Física

Es fundamental distinguir entre dos niveles de abstracción:

- **Arquitectura (ISA):** Es el modelo conceptual y programable del procesador visible para el desarrollador y el compilador. Define el juego de instrucciones, el repertorio y tamaño de los registros, el mapa de excepciones, los modos de direccionamiento y el modelo de memoria.
- **Implementación (Microarquitectura):** Es la realización circuital física en silicio. Incluye la cantidad de etapas del cauce (pipeline), la frecuencia máxima alcanzable, el tamaño de las memorias caché L1/L2, la tecnología litográfica de fabricación (ej. 28nm, 7nm) y el layout de los buses.

3.2. Comparación Detallada: Arquitecturas RISC vs CISC

Criterio	CISC (Complex Instruction Set Computer)	RISC (Reduced Instruction Set Computer)
Conjunto de Instrucciones	Extenso (cientos de instrucciones complejas y especializadas).	Reducido, simple y ortogonal (instrucciones primitivas elementales).
Tamaño de Instrucción	Variable (desde 1 byte hasta 15 bytes en x86). Decodificación compleja.	Fijo (32 bits en ARM, 16 bits en Thumb, 16/32 bits en Thumb-2). Decodificación rápida en 1 ciclo.
Acceso a Memoria	Las instrucciones aritméticas/lógicas pueden operar directamente sobre operandos en memoria.	Arquitectura Load/Store: Las operaciones aritmético-lógicas operan <i>exclusivamente</i> entre registros de la CPU. Solo LDR y STR tocan la memoria.
Banco de Registros	Pocos registros de uso general (históricamente 8 en x86 de 32 bits).	Gran banco de registros de propósito general (16 en Cortex-M: R0 a R15).
Complejidad del Circuito	Énfasis en el hardware (decodificadores microprogramados complejos).	Énfasis en el software y compilador (control cableado simple y rápido).

3.3. Evolución Histórica de las Versiones de la Arquitectura ARM

- **ARMv4:** Primer gran salto masivo comercial. Introduce el conjunto de instrucciones comprimidas **Thumb (16 bits)** para reducir el consumo de memoria Flash (núcleo clásico ARM7TDMI).
- **ARMv5:** Agrega instrucciones especializadas para Procesamiento Digital de Señales (DSP) y saturación aritmética.
- **ARMv6:** Introduce instrucciones SIMD (Single Instruction Multiple Data) para aceleración multimedia (núcleo ARM11).
- **ARMv6-M:** Versión minimalista de ultra bajo consumo diseñada para reemplazar microcontroladores de 8 y 16 bits (núcleos Cortex-M0 y Cortex-M0+).
- **ARMv7:** Punto de inflexión donde ARM divide formalmente su arquitectura en tres perfiles especializados:
 - **ARMv7-A (Application):** Con Unidad de Manejo de Memoria (MMU) para sistemas operativos de propósito general (Linux, Android) y extensiones vectoriales NEON.
 - **ARMv7-R (Real-Time):** Con Unidad de Protección de Memoria (MPU) para tiempo real crítico (frenos, aviónica).
 - **ARMv7-M (Microcontroller):** Diseñado específicamente para sistemas embebidos de control, introduce la tecnología **Thumb-2** (instrucciones mixtas de 16 y 32 bits con densidad de código óptima) y el controlador de interrupciones **NVIC integrado en el núcleo** (núcleos Cortex-M3 y Cortex-M4 con FPU).
- **ARMv8-A:** Introduce la arquitectura de 64 bits (AArch64) manteniendo compatibilidad con 32 bits.
- **ARMv8-M:** Incorpora extensiones de seguridad por hardware **TrustZone** al mundo de los microcontroladores (Cortex-M23, Cortex-M33).
- **ARMv9:** Añade extensiones vectoriales SVE2 y aceleradores de Inteligencia Artificial en el edge.

3.4. Plataforma de Estudio del Curso: STM32F103C8T6 ("Blue Pill")

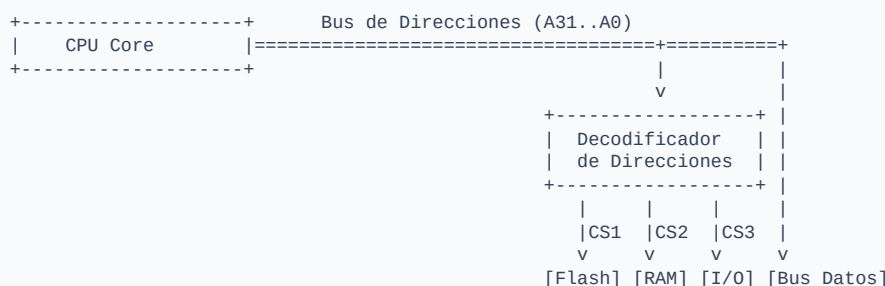
El hardware utilizado en la cátedra es el microcontrolador **STM32F103C8T6** de STMicroelectronics:

- **Núcleo:** ARM Cortex-M3 de 32 bits (arquitectura ARMv7-M) a frecuencia máxima de **72 MHz**.
- **Memoria:** 64 KiB de memoria Flash de programa y 20 KiB de memoria SRAM de datos.
- **Buses Internos:** Bus AHB (Advanced High-performance Bus) a 72 MHz, puenteado a dos buses periféricos:
 - **APB2 (Advanced Peripheral Bus 2):** Bus de alta velocidad a **72 MHz** (conecta GPIOs, ADC1, USART1, SPI1).
 - **APB1 (Advanced Peripheral Bus 1):** Bus de velocidad reducida a **36 MHz máximo** (conecta USART2, USART3, SPI2, I2C1, I2C2, Timers).
- **Toolchain Abierto:** Compilación mediante **GNU GCC** (`arm-none-eabi-gcc`), enlazado mediante Linker Scripts (`.ld`), y depuración en silicio mediante **GDB y OpenOCD** a través de interfaz SWD (Serial Wire Debug).

Capítulo 4: Mapeo de Memoria, Decodificador de Direcciones y Periféricos

4.1. El Decodificador de Direcciones y Chip Select

En un sistema digital de 32 bits, el procesador posee un bus de direcciones físico compartido que conecta en paralelo a todos los módulos de memoria (Flash, SRAM) y a todos los periféricos de entrada/salida. Para que un solo dispositivo responda en el bus de datos en un instante dado, se utiliza un **Decodificador de Direcciones**:



- El decodificador analiza los **bits más significativos (MSB)** del bus de direcciones.
- Genera señales de activación individuales denominadas **Chip Select (CS)** o **Chip Enable (CE)**, generalmente activas en nivel bajo (0 lógico).
- Cuando la CPU emite una dirección, solo el dispositivo cuyo rango coincide con los bits MSB ve activada su línea CS; todos los demás dispositivos colocan sus salidas de datos en **Alta Impedancia (Tri-State)** para evitar contención de bus.

Ejercicio Resuelto de Decodificación de Memoria (Típico de Parcial):

Se dispone de un procesador con bus de direcciones de 16 bits ($A_{15}..A_0$) y se desea decodificar 3 bloques contiguos de 16 KiB cada uno:

- Bloque 1 (EPROM de 16 KiB): Rango `0x0000` a `0x3FFF`.
- Bloque 2 (RAM de 16 KiB): Rango `0x4000` a `0x7FFF`.
- Bloque 3 (E/S de 16 KiB): Rango `0x8000` a `0xBFFF`.

Análisis de Direcciones:

16 KiB requieren 14 líneas de dirección ($2^{14} = 16384$ bytes, $A_{13}..A_0$). Por lo tanto, los dos bits superiores (A_{15}, A_{14}) definen el bloque:

Dispositivo	Rango Hexadecimal	A_{15}	A_{14}	Ecuación de Activación (CS activo en bajo)
EPROM	0x0000 - 0x3FFF	0	0	$\overline{CS_{EPROM}} = A_{15} + A_{14}$ (Compuerta OR)
RAM	0x4000 - 0x7FFF	0	1	$\overline{CS_{RAM}} = A_{15} + \overline{A_{14}}$
E/S	0x8000 - 0xBFFF	1	0	$\overline{CS_{IO}} = \overline{A_{15}} + A_{14}$

Implementación con Decodificador Comercial 74LS138 (Decodificador 3 a 8):

Se conectan las entradas de selección $C = 0$, $B = A_{15}$, $A = A_{14}$. Las salidas $\overline{Y_0}$, $\overline{Y_1}$ y $\overline{Y_2}$ corresponden directamente a $\overline{CS_{EPROM}}$, $\overline{CS_{RAM}}$ y $\overline{CS_{IO}}$.

4.2. Modelos de Entrada/Salida: Memory-Mapped I/O vs Port-Mapped I/O

Característica	Memory-Mapped I/O (Mapeada en Memoria)	Port-Mapped / Isolated I/O (Mapeada en Puertos)
Espacio de Direcciones	Único y Unificado: La memoria RAM, la Flash y los periféricos comparten el mismo espacio de 4 GiB (2^{32}).	Separado: Existe un espacio de memoria y un espacio de E/S aislado e independiente (ej. 64 KiB en x86).
Instrucciones de Acceso	Instrucciones estándar de memoria (LDR , STR en ARM; punteros en C).	Instrucciones especiales de máquina (IN y OUT en arquitectura x86).
Líneas de Control del Bus	Simples: Solo $\overline{RD}$ y $\overline{WR}$.	Señal adicional de control $M/\overline{IO}$ para indicar si la dirección corresponde a memoria o puerto.
Soporte en Lenguaje C	Total y nativo: Se accede a cualquier registro mediante punteros tipados y desreferenciación directa: <code>*(volatile uint32_t *)0x40010800 = 0x01;</code>	Requiere funciones intrínsecas del compilador o ensamblador embebido (ej. <code>inb()</code> , <code>outb()</code>).

Pregunta Clave de Examen: ¿Por qué la arquitectura ARM adopta Memory-Mapped I/O?

1.

Ortogonalidad del Conjunto de Instrucciones:

Permite manipular registros de hardware con todas las instrucciones de carga/almacenamiento, modos de direccionamiento indexados y desplazamientos de la ISA.

2.

Integración perfecta con el compilador de C:

Los registros de periféricos se modelan directamente como estructuras de C (structs) mapeadas a direcciones fijas mediante punteros constantes, sin requerir instrucciones especiales en el procesador.

4.3. El Registro como Abstracción de Hardware y el Uso de 'volatile'

Un **registro de periférico** no es una celda de memoria convencional; es un conjunto de **Flip-Flops (o latches)** conectados físicamente a las compuertas de control de un periférico. Escribir un bit '1' activa físicamente un transistor (ej. pone un pin en 3.3V); leer un bit consulta el estado de una señal externa del mundo real.

El Calificador 'volatile' en C:

Indica al compilador que el valor de esa dirección de memoria puede cambiar en cualquier momento debido a causas externas al software (un periférico o una interrupción). Prohíbe terminantemente al optimizador del compilador:

- Almacenar el valor en un registro de CPU para reutilizarlo en lecturas consecutivas (fuerza la lectura física en el bus cada vez).
- Eliminar escrituras aparentemente "redundantes" o bucles de espera por banderas de hardware (busy-wait loops como `while(! (USART1->SR & RXNE));`).

4.4. Clasificación y Jerarquía de Periféricos en ARM Cortex-M

En la arquitectura Cortex-M, los periféricos se dividen estrictamente en tres categorías según su nivel de integración:

```
+-----+
| 1. CORE PERIPHERALS (Internos al Núcleo) - Diseñados por ARM |
| Mapeados en el PRIVATE PERIPHERAL BUS (PPB): 0xE0000000 - 0xE00FFFFF |
| - NVIC (Nested Vectored Interrupt Controller) |
| - SysTick Timer (Temporizador del Kernel) |
| - SCB (System Control Block) & MPU (Memory Protection Unit) |
+-----+
| 2. ON-CHIP PERIPHERALS (Externos al Núcleo, Internos al Chip) - Diseñados por ST |
| Mapeados en buses AHB, APB1, APB2: 0x40000000 - 0x5FFFFFFF |
| - GPIOA..GPIOG, USART1..USART3, SPI1..SPI3, I2C1..I2C2, ADC1..ADC3, Timers |
+-----+
| 3. OFF-CHIP PERIPHERALS (Externos al Chip) - En el PCB del Producto Final |
| - Sensores I2C, Memorias Flash SPI, Transceptores RS-485, Displays LCD/OLED |
+-----+
```

Pregunta Clave de Examen: ¿Por qué los Periféricos del Núcleo (NVIC, SysTick) están mapeados en el Private Peripheral Bus (PPB) a partir de 0xE0000000?

1. Determinismo y Latencia Mínima:

El PPB es un bus de 32 bits privado y dedicado de la CPU. El tráfico del NVIC y SysTick

no compete jamás

por el ancho de banda con las transferencias de DMA ni con los periféricos de los buses AHB/APB.

2. Seguridad y Protección de Privilegios:

Las direcciones del PPB solo son accesibles en

Modo Privilegiado (Privileged Mode)

. Un hilo de usuario sin privilegios no puede deshabilitar interrupciones ni alterar el SysTick.

3. Portabilidad Absoluta (Estandarización ARM):

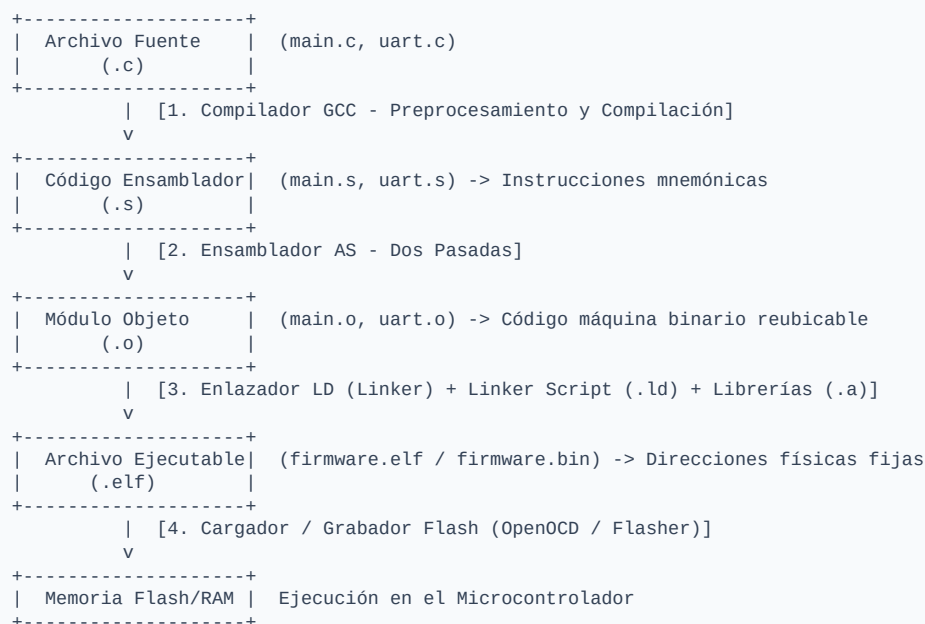
La ubicación de los registros del NVIC, SysTick y SCB es idéntica en cualquier microcontrolador Cortex-M del mundo (STM32, NXP LPC, TI MSPM0, Microchip SAM). Esto permite que el software de base (CMSIS y FreeRTOS) sea 100% reutilizable sin cambiar una sola línea de código de interrupciones.

MÓDULO 3: TOOLCHAIN, ENLAZADO Y ARRANQUE (BARE-METAL)

Capítulo 5: Flujo de Compilación, Ensamblado y Archivo Objeto (.o)

5.1. Las Cuatro Fases de la Cadena de Construcción (Toolchain)

Para transformar un conjunto de archivos fuente escritos en C en un ejecutable cargado en la memoria Flash de un microcontrolador, el software atraviesa cuatro etapas secuenciales:



5.2. Arquitectura Interna del Compilador GCC

El compilador moderno (GNU GCC) se estructura en tres bloques modulares:

1. **Front-End:** Específico del lenguaje de programación (C, C++). Realiza el análisis léxico (tokenización), sintáctico (gramática) y semántico, generando un Árbol de Sintaxis Abstracta (AST) y una representación intermedia inicial.
2. **Middle-End:** Independiente tanto del lenguaje fuente como de la arquitectura destino. Opera sobre el lenguaje intermedio **RTL (Register Transfer Language)** o GIMPLE. Aplica optimizaciones globales: eliminación de código muerto (dead code elimination), propagación de constantes, desenrollado de bucles (loop unrolling) e inlining de funciones.
3. **Back-End:** Específico de la arquitectura de hardware seleccionada (ARM Cortex-M3). Se encarga de la asignación óptima de registros físicos (R0 a R12), la selección de instrucciones máquina (Thumb-2), el ordenamiento del pipeline y la emisión del archivo en lenguaje ensamblador (`.s`).

5.3. El Ensamblador y el Algoritmo de las Dos Pasadas

El ensamblador toma el archivo `.s` y lo traduce a código binario. Debido a que en un programa existen **referencias hacia adelante (forward references)** (por ejemplo, un salto condicional a una etiqueta declarada 50 líneas más abajo), el ensamblador debe realizar **dos pasadas**:

- **Primera Pasada:** Lee el código línea por línea, actualiza un Contador de Ubicación relativo (Location Counter) y construye la **Tabla de Símbolos**. En esta tabla anota el nombre de cada etiqueta (funciones, variables) y le asigna su desplazamiento relativo (offset) dentro del archivo.
- **Segunda Pasada:** Relee el archivo fuente y genera el código máquina binario de cada instrucción. Reemplaza las etiquetas locales por sus desplazamientos calculados en la primera pasada y genera las **entradas de relocalización** para aquellos símbolos externos (definidos en otros archivos) que no pueden resolverse localmente.

5.4. Anatomía Exhaustiva del Archivo Objeto (.o / ELF)

El archivo `.o` está estructurado en formato **ELF (Executable and Linkable Format)** y contiene seis componentes fundamentales:

Sección / Componente	Tipo de Contenido	Destino en Memoria	Descripción Técnica
Cabecera (ELF Header)	Metadatos	No se carga	Identificador mágico ELF, arquitectura objetivo (ARM), endianness (Little-Endian), tamaño de palabras (32 bits) y tabla de offsets a secciones.
Segmento de Texto (.text)	Código y Constantes	Flash (ROM)	Instrucciones de código máquina binario ejecutables por la CPU, cadenas de texto literales (strings) y constantes declaradas con <code>const</code> . Permisos: Read-Only + Execute (rx) .
Segmento de Datos (.data)	Variables Globales Inicializadas	LMA: Flash VMA: RAM	Variables globales y estáticas cuyo valor inicial es <i>distinto de cero</i> (ej. <code>int contador = 100;</code>). Requiere valores en Flash al arrancar y espacio en RAM para modificarse.
Segmento BSS (.bss)	Variables Globales No Inicializadas	RAM (RW)	<i>Block Started by Symbol</i> . Variables globales y estáticas sin inicializar o inicializadas explícitamente a cero (ej. <code>uint8_t buffer[128];</code>). No ocupa espacio en el archivo binario Flash ; solo almacena el tamaño total requerido para que el Startup las limpie a cero en RAM.
Tabla de Símbolos (Symbol Table)	Metadatos de Nombres	No se carga	Lista de nombres de funciones, variables globales y estáticas, indicando su visibilidad (Global/Local/Weak), tipo, sección de pertenencia y offset.
Tabla de Relocalización (Relocation Info)	Punteros de Corrección	No se carga	Lista de instrucciones que contienen direcciones absolutas o llamadas a funciones externas que están incompletas y que el Linker debe parchear con direcciones fijas.

Capítulo 6: Enlazado (Linker), Linker Scripts (.ld), LMA vs VMA y Startup (crt.s)

6.1. Los Tres Pasos del Enlazador (Linker)

El Enlazador (Linker, `arm-none-eabi-ld`) combina múltiples archivos `.o` independientes y bibliotecas estáticas (`.a`) en un único archivo ejecutable monolítico:

- Colocación de Secciones:** Concatena y agrupa todas las secciones del mismo tipo de los distintos archivos objeto (une todos los `.text` en un gran `.text`, todos los `.data` y todos los `.bss`).
- Resolución de Símbolos:** Cruza las referencias cruzadas; asocia cada llamada a una función externa (ej. `USART_Init` invocada desde `main.o`) con la dirección de memoria exacta donde fue ubicada dicha función en `uart.o`.
- Relocalización (Patching de Direcciones):** Recorre las tablas de relocalización de cada archivo objeto y parchea los campos de dirección de las instrucciones binarias con las **direcciones físicas absolutas** asignadas según el mapa de memoria.

6.2. Mapa de Memoria del Sistema en Tiempo de Ejecución

```

Dirección Alta (0x20005000 - Fin de RAM en STM32F103)
+-----+
|          STACK          | <--- _esstack (Inicio inicial del SP)
|          |              | (Crece hacia abajo con llamadas a funciones,
|          v              | interrupciones y variables locales)
|          ^              |
|          |              |
|          HEAP           | (Crece hacia arriba con asignación dinámica)
+-----+
|          SECCIÓN .bss   | <--- _ebss
|                          | (RAM: Variables globales puestas a cero por Startup)
+-----+
|          SECCIÓN .data  | <--- _sbss / _edata
|                          | (RAM - VMA: Variables inicializadas con datos copiados de Flash)
+-----+
|                          | <--- _sdata (0x20000000 - Inicio de SRAM)
+-----+
=====
Dirección Alta Flash (0x08010000 - Fin de 64K Flash)
+-----+
|          VALORES INICIALES DE .data (LMA)          | (Copiados hacia RAM durante el arranque)
+-----+
|          SECCIÓN .text                             | <--- _sidata (LOADADDR(.data))
|                          | (Flash: Código binario ejecutable y constantes)
+-----+
|          TABLA DE VECTORES DE EXCEPCIÓN             | (Flash: Offset 0x00: Initial SP, Offset 0x04: Reset_Handler)
+-----+
|          0x08000000 (Inicio de Flash / Alias 0x00000000)
+-----+
Dirección Baja

```

6.3. El Linker Script (.ld) en Detalle

El Linker Script es el archivo de configuración que le enseña al enlazador la arquitectura física de la memoria del chip. Se compone de dos bloques obligatorios:

1. El Bloque MEMORY

Define las regiones de memoria física reales, sus atributos de acceso, su dirección de inicio (`ORIGIN`) y su tamaño total (`LENGTH`):

```
MEMORY
{
    FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 64K /* Flash de sólo lectura y ejecución */
    RAM (rwx) : ORIGIN = 0x20000000, LENGTH = 20K /* SRAM de lectura, escritura y ejecución */
}
/* Símbolo que calcula la cima de la pila (Stack Pointer inicial) */
_esstack = ORIGIN(RAM) + LENGTH(RAM); /* 0x20000000 + 20*1024 = 0x20005000 */
```

2. El Bloque SECTIONS

Indica cómo mapear las secciones de los archivos objeto en las memorias físicas:

```
SECTIONS
{
    /* 1. Tabla de vectores en el primer byte de Flash */
    .isr_vector :
    {
        . = ALIGN(4);
        KEEP(*(.isr_vector)) /* KEEP impide que el linker la descarte por falta de uso */
        . = ALIGN(4);
    } > FLASH

    /* 2. Código ejecutable y constantes */
    .text :
    {
        . = ALIGN(4);
        *(.text) /* Todo el código de todos los .o */
        *(.text*)
        *(.rodata) /* Datos de solo lectura (constantes, strings) */
        *(.rodata*)
        . = ALIGN(4);
        _etext = .; /* Símbolo que marca el fin de código en Flash */
    } > FLASH

    /* 3. Sección .data con LMA en Flash y VMA en RAM */
    _sidata = LOADADDR(.data); /* LMA: Dirección física en Flash donde residen los datos */

    .data :
    {
        . = ALIGN(4);
        _sdata = .; /* VMA: Dirección de inicio de .data en RAM */
        *(.data)
        *(.data*)
        . = ALIGN(4);
        _edata = .; /* VMA: Dirección de fin de .data en RAM */
    } > RAM AT > FLASH /* VMA en RAM, pero cargado (LMA) en FLASH */

    /* 4. Sección .bss solo en RAM */
    .bss :
    {
        . = ALIGN(4);
        _sbss = .; /* Dirección de inicio de .bss en RAM */
        *(.bss)
        *(.bss*)
        *(COMMON)
        . = ALIGN(4);
        _ebss = .; /* Dirección de fin de .bss en RAM */
    } > RAM
}
```


CONCEPTO VITAL DE EXAMEN: La Dualidad LMA vs VMA en la Sección .data

Una variable global inicializada (ej. `int umbral = 75;`) presenta una contradicción fundamental en un sistema embebido:

- **Debe ser No Volátil:**

Su valor inicial (75) debe sobrevivir al corte de energía, por lo que *debe almacenarse físicamente en la memoria Flash*.

- **Debe ser Modificable:**

Durante la ejecución el programa puede alterar su valor (ej. `umbral = 80;`), pero la memoria Flash no se puede sobrescribir con instrucciones normales de CPU en tiempo de ejecución. Por ende, la variable *debe residir y ejecutarse en memoria RAM*.

La Solución:

- **LMA (Load Memory Address):**

Es la dirección de almacenamiento persistente en

Flash

, obtenida con `LOADADDR(.data)` y exportada al símbolo `__sdata`.

- **VMA (Virtual / Execution Memory Address):**

Es la dirección de ejecución en

RAM

(marcada entre `__sdata` y `__edata`).

6.4. El Archivo de Arranque (Startup crt.s / startup.c)

Cuando el microcontrolador sale de un estado de Reset de hardware, el procesador no salta mágicamente a `main()`. El archivo de arranque (C Runtime Startup, `crt.s`) es el primer fragmento de software en ejecutarse y cumple cinco funciones obligatorias:

1. **Instanciar la Tabla de Vectores:** Coloca en la dirección `0x08000000` el valor inicial de la cima de la pila (`__esstack`) y en la dirección `0x08000004` la dirección de la primera instrucción: `Reset_Handler`.
2. **Copiar la Sección .data de Flash a RAM:** Transfiere byte a byte los valores iniciales desde la dirección LMA (`__sdata` en Flash) hacia la dirección VMA (desde `__sdata` hasta `__edata` en RAM).
3. **Inicializar la Sección .bss a Cero (Zero-Fill):** Escribe ceros (`0x00`) en todo el rango de memoria RAM delimitado entre `__sbss` y `__ebss`. Esto garantiza el estándar de C que exige que toda variable global no inicializada valga cero al inicio.
4. **Llamar a Funciones de Inicialización de Hardware:** Invoca opcionalmente a `SystemInit()` para configurar el circuito de reloj y PLL a 72 MHz.
5. **Transferir el Control a la Aplicación:** Ejecuta una instrucción de salto `bl main`.

Código Completo Comentado del Reset_Handler:

```
void Reset_Handler(void)
{
    // Símbolos provistos por el Linker Script (.ld)
    extern uint32_t _sidata; // Dirección de inicio en Flash de los datos
    extern uint32_t _sdata; // Dirección de inicio en RAM de la sección .data
    extern uint32_t _edata; // Dirección de fin en RAM de la sección .data
    extern uint32_t _sbss;   // Dirección de inicio en RAM de la sección .bss
    extern uint32_t _ebss;   // Dirección de fin en RAM de la sección .bss

    // 1. Copia de la sección .data (LMA -> VMA)
    uint32_t *pSrc = &_sidata;
    uint32_t *pDst = &_sdata;
    while (pDst < &_edata) {
        *pDst++ = *pSrc++;
    }

    // 2. Limpieza a cero de la sección .bss
    uint32_t *pBss = &_sbss;
    while (pBss < &_ebss) {
        *pBss++ = 0;
    }

    // 3. Inicialización del reloj del sistema
    SystemInit();

    // 4. Salto a la función principal main()
    main();

    // Si main() retorna por error, atrapar la CPU en un bucle infinito
    while (1);
}
```

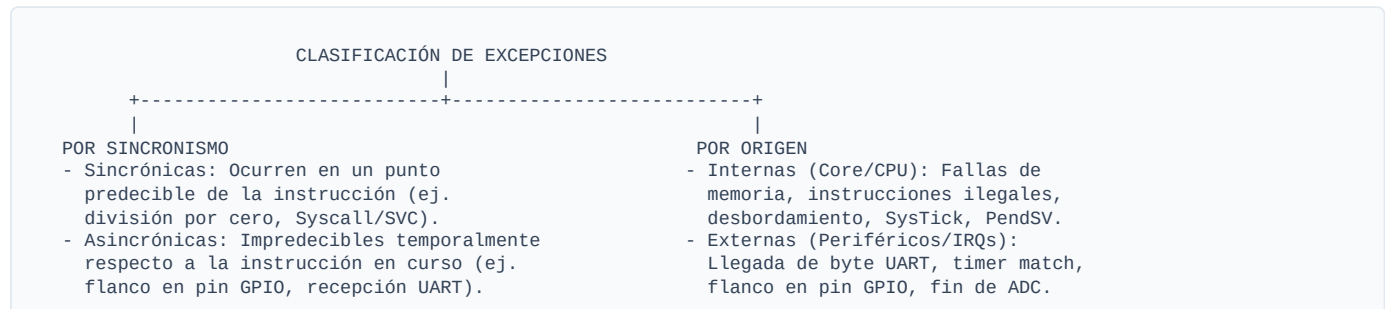
MÓDULO 4: CONTROLADOR DE INTERRUPCIONES Y EXCEPCIONES (NVIC)

Capítulo 7: Concepto y Clasificación de Excepciones

7.1. Definición Formal de Excepción

Una **excepción** es cualquier evento no programado que altera el flujo secuencial de ejecución de instrucciones del procesador, forzando a la CPU a suspender la tarea en curso y ejecutar una rutina de servicio especializada (**ISR - Interrupt Service Routine** o Exception Handler).

7.2. Clasificación Dual de las Excepciones



7.3. Acciones Atómicas del Hardware en el Datapath

Ante la ocurrencia de cualquier excepción, el hardware del procesador ejecuta tres acciones atómicas e indivisibles:

1. **Guardar la Dirección de Retorno:** Salva la dirección de la siguiente instrucción a ejecutar en el registro de enlace (`LR` / Stack) para poder reanudar el programa interrumpido exactamente donde quedó.
2. **Registrar la Causa del Evento:** Almacena el número o tipo de excepción en registros de estado internos (ej. registro `IPSR` del núcleo).
3. **Forzar el Salto al Manejador:** Carga en el Program Counter (`PC`) la dirección de inicio de la rutina de atención (ISR).

7.4. Identificación de Causas: Polling vs Interrupciones Vectorizadas

- **Mecanismo por Sondeo (Polled Interrupts - Registro de Causa):** Utilizado en microprocesadores antiguos (ej. MIPS básico o 8080). Todas las interrupciones saltan a una *única dirección fija común*. Un software intermediario debe leer secuencialmente múltiples registros de estado mediante `if()` sucesivos para descubrir qué periférico causó la interrupción, generando un retardo (latencia de interrupción) elevado y no determinista.
- **Mecanismo Vectorizado (Vectored Interrupts - Usado en ARM NVIC):** Existe una **Tabla de Vectores** en memoria donde cada fuente de interrupción tiene asignada una posición exclusiva de 32 bits que contiene el puntero directo a su ISR. El hardware del NVIC decodifica la causa, calcula el índice y realiza el salto directamente a la función correspondiente en tiempo récord y constante.

Capítulo 8: El NVIC en ARM Cortex-M3 en Profundidad

8.1. Características Distintivas del NVIC

El **NVIC (Nested Vectored Interrupt Controller)** es el controlador de excepciones diseñado por ARM e integrado físicamente dentro del silicio del núcleo Cortex-M3:

- **Nested (Anidado / Pre-emption):** Soporta anidamiento por hardware. Una interrupción de mayor prioridad puede interrumpir y desalojar inmediatamente a una ISR de menor prioridad que se encuentre en plena ejecución.
- **Vectored (Vectorizado):** Salto directo indexado por hardware sin intermediación de software de despacho.
- **Baja Latencia Fija y Determinista:** La transición desde la llegada del pulso de interrupción hasta el inicio de la primera instrucción de la ISR toma exactamente **12 ciclos de reloj**.
- **Técnicas Avanzadas de Reducción de Latencia:**
 - **Tail-Chaining:** Si una interrupción de igual o menor prioridad queda pendiente mientras termina una ISR activa, el procesador *no desapila ni vuelve a apilar* los registros; encadena directamente la ejecución de la siguiente ISR en solo **6 ciclos de reloj**.

- **Late-Arrival (Llegada Tardía):** Si durante el apilamiento de registros de una interrupción de baja prioridad aparece una interrupción de alta prioridad, el hardware redirige el vector fetch hacia la de mayor prioridad sin reiniciar el ciclo de stacking.

8.2. Tabla de Numeración de Excepciones en Cortex-M

Nº Excepción	Tipo / Nombre	Prioridad	CMSIS IRQn	Descripción Técnica
1	Reset	-3 (Fija)	-15	Reinicio por hardware o caída de tensión (Power-On).
2	NMI (Non-Maskable Interrupt)	-2 (Fija)	-14	Interrupción no enmascarable (fallo de reloj, brownout).
3	HardFault	-1 (Fija)	-13	Falla irrecuperable de hardware o escalada de faltas no atendidas.
4	MemManage	Programable	-12	Violación de permisos de memoria por la MPU.
5	BusFault	Programable	-11	Error de respuesta en el bus de direcciones o datos (ej. acceso a periférico apagado).
6	UsageFault	Programable	-10	Instrucción no definida, división por cero, alineación no válida.
11	SVCall	Programable	-5	SuperVisor Call: Syscall para invocar al kernel desde modo no privilegiado.
14	PendSV	Programable	-2	Pendable Service: Excepción asincrónica para cambio de contexto en RTOS.
15	SysTick	Programable	-1	Temporizador del núcleo para generar los ticks del sistema operativo.
16 a 255	IRQ0 a IRQ239	Programable	0 a 239	Interrupciones de periféricos externos (USART1, TIM2, SPI1, EXTI, ADC).

Regla CMSIS IRQn:

Las excepciones internas del sistema ARMv7-M tienen enumeraciones **negativas**

(desde -15 hasta -1). Las interrupciones externas de periféricos del fabricante comienzan en **cero (0)** en adelante.

8.3. Esquema de Prioridades: Pre-emption (Grupo) vs Sub-prioridad

El estándar ARM Cortex-M define registros de prioridad de 8 bits por cada interrupción programable. La convención es estricta: **menor valor numérico representa mayor prioridad** (el valor 0 es la máxima prioridad programable; 255 es la menor).

El registro **AIRCR** (Application Interrupt and Reset Control Register) del SCB contiene el campo **PRIGROUP**, el cual divide los bits de prioridad de cada interrupción en dos partes:

Registro de Prioridad de 8 bits (IPR)

+-----+-----+	
PRE-EMPTION PRIORITY (Grupo)	SUB-PRIORITY
+-----+-----+	

- **Pre-emption Priority (Prioridad de Grupo / Desalojo):** Determina si una nueva interrupción puede **interrumpir y desalojar** a una interrupción que ya se está ejecutando. Una ISR solo puede ser desalojada por otra cuyo campo de grupo sea *numéricamente menor*.
- **Sub-prioridad:** Se utiliza exclusivamente como **criterio de desempate** cuando dos o más interrupciones que pertenecen al *mismo grupo de prioridad* llegan en forma **SIMULTÁNEA**. La que posea menor sub-prioridad se atenderá primero.

TRAMPA CLÁSICA DE PARCIAL:

Si una interrupción con Grupo = 2, Sub-prioridad = 0 está ejecutándose, y llega otra interrupción con Grupo = 2, Sub-prioridad = 1,

NO se produce desalojo

(no se anida), porque tienen el *mismo nivel de grupo*. La segunda interrupción quedará en estado Pending y esperará a que la primera termine completamente.

Configuración con CMSIS:

```
// Para modificar AIRCR se requiere la clave VECTKEY (0x05FA en los 16 bits superiores)
// Ejemplo: Configurar 2 bits de Grupo (4 niveles) y 2 bits de Subprioridad (4 niveles)
NVIC_SetPriorityGrouping(NVIC_PRIORITYGROUP_2);

// Asignar Prioridad a USART1 (Grupo 1, Sub-prioridad 0)
uint32_t prio = NVIC_EncodePriority(NVIC_PRIORITYGROUP_2, 1, 0);
NVIC_SetPriority(USART1_IRQn, prio);

// Habilitar la interrupción en el NVIC
NVIC_EnableIRQ(USART1_IRQn);
```

8.4. Reubicación de la Tabla de Vectores (SCB->VTOR)

Por defecto, la tabla de vectores reside en la dirección 0x00000000 (mapeada a Flash en 0x08000000). Para sistemas con **Bootloader** o cuando se requiere modificar dinámicamente vectores en tiempo de ejecución, la tabla puede reubicarse a la memoria SRAM utilizando el registro **VTOR (Vector Table Offset Register)** del System Control Block (SCB):

```
// Reubicar tabla de vectores a SRAM (0x20000000)
__disable_irq();
SCB->VTOR = 0x20000000;
__DSB(); // Data Synchronization Barrier
__ISB(); // Instruction Synchronization Barrier
__enable_irq();
```

Regla de Alineación Estricta del VTOR:

La dirección base asignada a VTOR debe estar alineada a la

potencia de 2 inmediatamente superior al tamaño total de la tabla

. Para STM32F103 (16 excepciones de sistema + 43 IRQs = 59 vectores de 4 bytes = 236 bytes), la potencia de 2 siguiente es **256 bytes**

. Por lo tanto, los 8 bits inferiores de VTOR deben ser obligatoriamente 0x00 (ej. 0x20000100, 0x20000200).

8.5. Estados de una Interrupción en el NVIC

Una línea de interrupción atraviesa tres estados lógicos independientes en el hardware:

1. **Disabled / Enabled:** Habilitada o deshabilitada mediante los registros **ISER** (Interrupt Set-Enable) e **ICER** (Interrupt Clear-Enable).
2. **Pending (Pendiente):** El evento ocurrió y está a la espera de ser atendido por la CPU (registros **ISPR** e **ICPR**). Si la CPU está ocupada con una interrupción de mayor prioridad o las interrupciones están enmascaradas, la bandera Pending permanece en 1.
3. **Active (Activa):** La CPU se encuentra ejecutando actualmente las instrucciones de la ISR correspondiente (reflejado en el registro **IABR** - Interrupt Active Bit Register).

Diseño Atómico de Registros ISER / ICER:

El NVIC separa deliberadamente la habilitación en dos registros independientes (**ISER** para poner en 1 y **ICER** para poner en 0). Esto **elimina las condiciones de carrera (Race Conditions)**

del clásico ciclo de software "Lectura-Modificación-Escritura" (Read-Modify-Write) en sistemas multitarea.

Capítulo 9: Auto-Stacking por Hardware, EXC_RETURN y Retorno de Excepción

9.1. Secuencia de Entrada a la Excepción (Exception Entry Sequence)

Cuando el NVIC acepta una solicitud de interrupción, el hardware del núcleo Cortex-M3 ejecuta en paralelo y de forma totalmente automática la siguiente secuencia en 12 ciclos de reloj:

STACK ACTIVO (MSP o PSP)		
	xPSR	<--- Offset +28
	PC	<--- Offset +24 (Dirección de retorno)
	LR	<--- Offset +20
	R12	<--- Offset +16
	R3	<--- Offset +12
	R2	<--- Offset +8
	R1	<--- Offset +4
SP nuevo ->	R0	<--- Offset +0

1. **Hardware Auto-Stacking (Apilamiento Automático):** El hardware empuja en el Stack activo (MSP o PSP) un marco de **8 registros (32 bytes)**: R0, R1, R2, R3, R12, LR, PC, xPSR.

¿Por qué estos registros específicos? Porque según el estándar ABI de ARM (AAPCS), estos registros son los denominados "caller-saved". Al guardarlos automáticamente el hardware, **cualquier función estándar escrita en C puede utilizarse directamente como ISR** sin necesidad de palabras clave especiales ni código assembly previo.

2. **Vector Fetch en Paralelo:** Mientras el hardware apila los registros por el bus de datos, el bus de instrucciones lee de la Tabla de Vectores la dirección de inicio de la ISR.

3. **Carga del Valor Mágico EXC_RETURN en LR:** El registro LR (Link Register) es sobrescrito con un valor especial con patrón 0xFFFFFFFx (ej. 0xFFFFFFF9 o 0xFFFFFFF0), que codifica el estado previo del procesador (modo y stack utilizado).

4. **Transición de Modo:** La CPU conmuta automáticamente a **Handler Mode** (modo de interrupción, que es *siempre Privilegiado* y *utiliza obligatoriamente el Main Stack Pointer - MSP*).

5. **Salto a la ISR:** El PC se carga con la dirección obtenida en el Vector Fetch y comienza la ejecución.

9.2. Secuencia de Retorno de Excepción (Exception Return Sequence)

Para finalizar una ISR, el código ejecuta una instrucción de retorno estándar como `bx lr` o `pop {pc}`.

	1. La CPU intenta cargar el valor de LR (EXC_RETURN: 0xFFFFFFFx) en el PC.	
	2. La lógica del procesador DETECTA el patrón mágico 0xFFFFFFFx.	
	(No realiza un salto a esa dirección de memoria).	
	3. Hardware Unstacking Automático:	
	- Desapila R0, R1, R2, R3, R12, LR, PC y xPSR del Stack correspondiente.	
	- Restaura el modo previo (Thread Mode con MSP o PSP).	
	- Limpia el bit de interrupción Activa en el registro IABR del NVIC.	
	4. La CPU reanuda la ejecución en la dirección exacta almacenada en el PC	
	desapilado, con las banderas del xPSR totalmente restauradas.	

9.3. Registros de Enmascaramiento Global del Núcleo

Registro Especial	Instrucción Ensamblador / CMSIS	Comportamiento y Nivel de Enmascaramiento
PRIMASK	CPSID i / __disable_irq() CPSIE i / __enable_irq()	Registro de 1 bit. Al ponerse en 1, eleva la prioridad del procesador a 0 . Enmascara y bloquea <i>todas las interrupciones programables</i> (IRQs y excepciones con prioridad ≥ 0). Solo permite la ejecución de Reset, NMI y HardFault.
FAULTMASK	CPSID f / CPSIE f	Registro de 1 bit. Eleva la prioridad del procesador a -1 . Enmascara todas las interrupciones programables y también la falta HardFault. Solo permite la ejecución de NMI y Reset.
BASEPRI	MSR BASEPRI, R0 __set_BASEPRI(prioridad)	Registro de 8 bits. Permite enmascarar interrupciones de forma selectiva por umbral . Al escribir un valor <i>N</i> , se bloquean todas las excepciones con prioridad <i>numéricamente mayor o igual a N</i> , permitiendo que las interrupciones críticas con prioridad superior (menor a <i>N</i>) sigan ejecutándose. Utilizado extensivamente por FreeRTOS para sus Secciones Críticas.

MÓDULO 5: PERIFÉRICOS Y PROTOCOLOS DE COMUNICACIÓN SERIE

Capítulo 10: Metodología de Activación y Configuración de GPIO / RCC

10.1. Las Cuatro Capas Jerárquicas de Activación de un Periférico

En microcontroladores modernos como STM32, ningún periférico funciona por defecto al reiniciar el chip (para ahorrar energía). La inicialización debe seguir rigurosamente cuatro capas jerárquicas:

```
+-----+
| 1. CAPA DE INFRAESTRUCTURA (RCC - Reset and Clock Control) |
|   Habilita la señal de reloj en el bus correspondiente (AHB, APB1 o APB2). |
|   ¡Sin reloj, los registros del periférico permanecen inaccesibles o congelados! |
+-----+
| 2. CAPA DE INTERFAZ FÍSICA (GPIO / AFIO) |
|   Configura los pines físicos (Modo Entrada, Salida, Función Alternativa) y |
|   activa el multiplexor de remapeo (AFIO) si los pines están reubicados. |
+-----+
| 3. CAPA DE PROTOCOLO / CONTROL |
|   Configura los registros de control específicos del periférico: velocidad de |
|   baudios, polaridad, paridad, tamaño de palabra (8/16b), divisores y modos. |
+-----+
| 4. CAPA OPERATIVA / ENLACE |
|   Enciende funcionalmente el periférico activando su bit de Enable |
|   (ej. bit SPE en SPI, bit UE en USART, bit ADON en ADC). |
+-----+
```

10.2. Distribución de Buses en STM32F103

- **APB2 (Alta Velocidad - 72 MHz máx):** Registros de control de reloj `RCC->APB2ENR`. Conecta: **GPIOA, GPIOB, GPIOC, GPIOD, GPIOE, AFIO, ADC1, ADC2, TIM1, SPI1, USART1**.
- **APB1 (Baja Velocidad - 36 MHz máx):** Registros de control de reloj `RCC->APB1ENR`. Conecta: **TIM2, TIM3, TIM4, USART2, USART3, SPI2, I2C1, I2C2, CAN, USB**.

10.3. Configuración de Puertos GPIO en STM32F103

Cada puerto GPIO dispone de 16 pines (0 a 15). Su comportamiento se configura mediante dos registros de 32 bits, donde **cada pin ocupa exactamente 4 bits** de configuración:

- **GPIOx_CRL (Control Register Low):** Configura los pines **0 a 7**.
- **GPIOx_CRH (Control Register High):** Configura los pines **8 a 15**.

Estructura de 4 bits por pin: [CNF1 | CNF0 | MODE1 | MODE0]

Modo (MODE[1:0])	Configuración (CNF[1:0])	Valor Binario (4 bits)	Valor Hex	Descripción Funcional y Aplicación Típica
00: Modo Entrada (Entrada digital o analógica)	00: Entrada Analógica	0000	0x0	Pin conectado directamente al ADC. Desconecta buffers Schmitt digitales (mínimo consumo).
	01: Entrada Flotante	0100	0x4	Entrada digital de alta impedancia (Hi-Z). Estado por defecto tras Reset (ej. RX UART).
	10: Entrada con Pull-Up / Pull-Down	1000	0x8	Resistencia interna activada. Se selecciona Pull-Up escribiendo 1 en GPIOx->ODR o Pull-Down escribiendo 0 en GPIOx->ODR.
Salida: 01: 10 MHz 10: 2 MHz 11: 50 MHz (Máx)	00: Propósito General Push-Pull	0011 (50MHz)	0x3	Salida digital normal que conmuta activamente entre 0V (GND) y 3.3V (VDD) (ej. encender un LED).
	01: Propósito General Open-Drain	0111 (50MHz)	0x7	Drenador abierto. Solo conmuta a GND (0V) o queda flotante (Hi-Z). Requiere pull-up externa.
	10: Función Alternativa Push-Pull	1011 (50MHz)	0xB	El pin es controlado por un periférico interno en modo activo (ej. TX de USART1, SCK/MOSI de SPI1).
	11: Función Alternativa Open-Drain	1111 (50MHz)	0xF	El pin es controlado por un periférico en drenador abierto (ej. SDA y SCL de I2C1).

Ejemplo Práctico de Configuración de GPIO en C (USART1 en PA9 y PA10):

```
// 1. Habilitar relojes de GPIOA y USART1 en APB2
RCC->APB2ENR |= RCC_APB2ENR_IOPAEN | RCC_APB2ENR_USART1EN;

// 2. Configurar PA9 (TX) como Función Alternativa Push-Pull 50MHz (0xB)
// y PA10 (RX) como Entrada Flotante (0x4)
// Como son pines 9 y 10, se utiliza GPIOA->CRH
GPIOA->CRH &= ~(0xFF << 4); // Limpiar bits de PA9 (pos 4) y PA10 (pos 8)
GPIOA->CRH |= (0x4B << 4); // PA10 = 0x4 (bits 11:8), PA9 = 0xB (bits 7:4)
```

Capítulo 11: Protocolo SPI (Serial Peripheral Interface)

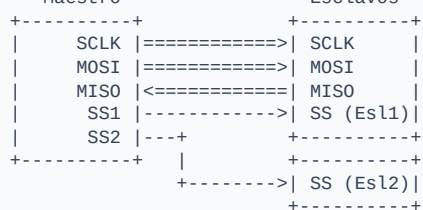
11.1. Principios del Protocolo SPI

Desarrollado por Motorola, SPI es una interfaz de comunicación serie **síncrona, full-duplex y maestro-esclavo** de 4 hilos:

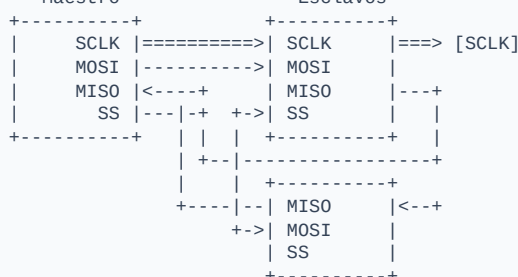
- **SCLK (Serial Clock):** Generado exclusivamente por el Maestro para sincronizar cada bit transmitido.
- **MOSI (Master Output, Slave Input):** Línea de datos desde el maestro hacia los esclavos.
- **MISO (Master Input, Slave Output):** Línea de datos desde el esclavo hacia el maestro.
- **SS / NSS (Slave Select / Not Slave Select):** Señal activa en nivel bajo (0) para habilitar un esclavo individual.

11.2. Topologías de Red SPI

1. ESCLAVOS INDEPENDIENTES (Star Topology)



2. CADENA EN CASCADA (Daisy-Chain Topology)



11.3. Modos de Reloj: Polaridad (CPOL) y Fase (CPHA)

La sincronización exacta de datos se define mediante dos parámetros en el registro de control:

- **CPOL (Clock Polarity):**
 - **CPOL = 0** : La línea de reloj SCLK permanece en **nivel bajo (0)** durante el estado de reposo (idle).
 - **CPOL = 1** : La línea de reloj SCLK permanece en **nivel alto (1)** durante el estado de reposo (idle).
- **CPHA (Clock Phase):**
 - **CPHA = 0** : El dato es **muestreado (leído) en el PRIMER flanco** de reloj. El dato debe presentarse antes (cuando la línea SS cae a bajo). *Requiere levantar y bajar SS entre bytes sucesivos.*
 - **CPHA = 1** : El dato es **muestreado (leído) en el SEGUNDO flanco** de reloj. El dato se presenta (cambia) en el primer flanco. Permite transmisiones continuas de múltiples bytes.

Modo SPI	CPOL	CPHA	Estado de Reposo	Flanco de Muestreo (Lectura)	Flanco de Transición (Cambio)
Modo 0	0	0	Bajo (0V)	Primer flanco (Ascendente)	Segundo flanco (Descendente)
Modo 1	0	1	Bajo (0V)	Segundo flanco (Descendente)	Primer flanco (Ascendente)
Modo 2	1	0	Alto (3.3V)	Primer flanco (Descendente)	Segundo flanco (Ascendente)
Modo 3	1	1	Alto (3.3V)	Segundo flanco (Ascendente)	Primer flanco (Descendente)

11.4. Gestión de NSS en STM32: Software vs Hardware

- **Gestión por Software (SSM = 1):** El pin físico NSS queda libre para uso general. El nivel del pin interno se fuerza mediante el bit **SSI** en **SPI_CR1**. En modo maestro, se coloca **SSM=1** y **SSI=1**, y se controla cualquier pin GPIO normal por software para seleccionar al esclavo.
- **Gestión por Hardware (SSM = 0):**
 - **SSOE = 1** (Master Output Enable): El microcontrolador comanda automáticamente el pin NSS a nivel bajo mientras transmite y lo libera al terminar.
 - **SSOE = 0** (Modo Multimaestro / Esclavo): Si otro dispositivo externo pone el pin NSS a nivel bajo, el módulo detecta una colisión de bus (Master Mode Fault - MODF) y se conmuta automáticamente a modo esclavo para proteger el circuito.

11.5. Código Completo en C de Transmisión/Recepción SPI en STM32

```
// Función bloqueante Full-Duplex de 16 bits en SPI1
uint16_t SPI1_TransmitReceive16(uint16_t dataOut)
{
    // 1. Esperar a que el buffer de transmisión esté vacío (TXE = 1)
    while (!(SPI1->SR & SPI_SR_TXE));

    // 2. Escribir el dato en el registro de datos (inicia la generación de reloj)
    SPI1->DR = dataOut;

    // 3. Esperar a que el buffer de recepción contenga el dato leído (RXNE = 1)
    while (!(SPI1->SR & SPI_SR_RXNE));

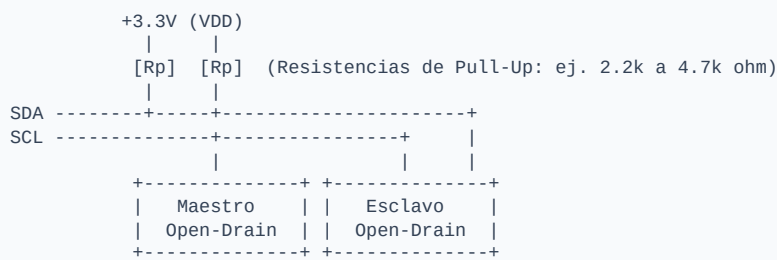
    // 4. Leer y retornar el dato recibido desde el esclavo
    return (uint16_t)SPI1->DR;
}
```

Capítulo 12: Protocolo I2C (Inter-Integrated Circuit)

12.1. Principios Eléctricos del Protocolo I2C

Desarrollado por Philips en 1982, I2C es un bus serie **síncrono**, **half-duplex**, **multimaestro** de solo 2 líneas bidireccionales:

- **SDA (Serial Data):** Línea bidireccional para transmisión de direcciones y datos.
- **SCL (Serial Clock):** Línea de reloj generada por el maestro que comanda el bus en ese instante.



Topología Eléctrica en Drenador Abierto (Open-Drain): Ningún nodo conectado al bus posee un transistor que fuerce la línea a nivel alto (VDD). Todos los dispositivos operan en drenador abierto: solo pueden *llevar la línea a masa (0V)* o dejarla *flotante (Hi-Z)*. El estado de reposo en nivel alto es garantizado pasivamente por las resistencias de Pull-Up (R_p).

12.2. Arbitraje Multimaestro y Lógica "Wired-AND"

Debido a la topología Open-Drain, si un nodo coloca un '0' lógico (conduce a GND), la línea cae a 0V para **todos los dispositivos del bus** (conexión AND cableada).

Mecanismo de Arbitraje sin Pérdida de Datos:

Si dos maestros intentan transmitir simultáneamente, ambos monitorean la línea SDA mientras transmiten. En el primer bit en que el Maestro 1 intenta transmitir un '1' (deja flotante SDA) pero el Maestro 2 transmite un '0' (lleva SDA a GND), el Maestro 1 lee '0' en SDA. Al detectar que lo que lee no coincide con lo que transmitió, el Maestro 1

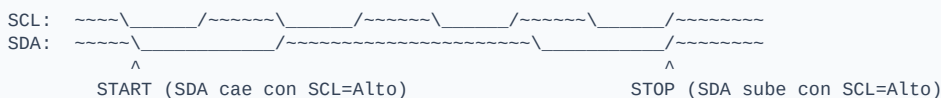
pierde inmediatamente el arbitraje

y desconecta sus salidas sin corromper la trama del Maestro 2, que continúa su transmisión sin enterarse de la colisión.

12.3. Modos de Velocidad I2C

Modo	Abreviatura	Velocidad Máxima	Características Técnicas
Standard Mode	Sm	100 kbps	Modo clásico original, pull-up resistivo estándar.
Fast Mode	Fm	400 kbps	Tiempos de subida más rápidos, pendiente controlada.
Fast Mode Plus	Fm+	1 Mbps	Requiere transistores de mayor corriente (20 mA) para bajar la línea rápidamente.
High-Speed Mode	Hs	3.4 Mbps	Usa fuentes de corriente activa en el pull-up y resistencias serie (R_s) para atenuar reflexiones.
Ultra-Fast Mode	UFm	5 Mbps	Unidireccional exclusivo, utiliza salidas Push-Pull sin ACK/NACK (no compatible con I2C clásico).

12.4. Anatomía de la Trama y Estados del Protocolo I2C



- **Condición de START:** Transición de nivel **ALTO a BAJO** en la línea SDA mientras SCL permanece en **ALTO**.
- **Condición de STOP:** Transición de nivel **BAJO a ALTO** en la línea SDA mientras SCL permanece en **ALTO**.
- **Regla de Validez de Datos:** Durante la transmisión de bits normales, SDA solo puede cambiar de estado cuando SCL está en nivel **BAJO**. El dato se considera válido y estable para lectura exclusivamente cuando SCL está en nivel **ALTO**.
- **Byte de Dirección:** Los primeros 7 bits representan la dirección única del esclavo; el 8vo bit es el bit de dirección de flujo **R/W** (0 = Escritura hacia el esclavo, 1 = Lectura desde el esclavo).
- **Bit de ACK / NACK (9no pulso de reloj):** Tras recibir cada byte, el emisor libera la línea SDA (la deja en Hi-Z). Durante el 9no pulso de SCL:
 - **ACK (Acknowledge = 0):** El receptor lleva activamente SDA a nivel bajo (0V), confirmando recepción exitosa.

- ## Capítulo 13: Protocolo UART / USART y Estándar RS-232

Ecuación de Configuración del Baud Rate:

$$\text{Baud Rate} = \frac{f_{\text{CK}}}{16 \times \text{USARTDIV}}$$

El valor de `USARTDIV` se desglosa en el registro de 16 bits `USART_BRR` en formato de punto fijo:

- `DIV_Mantissa[15:4]` : Parte entera (12 bits).
- `DIV_Fraction[3:0]` : Parte fraccionaria escalada por 16 (4 bits).

Cálculo Resuelto Paso a Paso (Pregunta Clásica de Examen):

Calcular el valor exacto a cargar en `USART1->BRR` para operar a

9600 baudios

con el microcontrolador a

72 MHz

(USART1 en APB2, $f_{\text{CK}} = 72 \text{ MHz}$):

1. $\text{USARTDIV} = \frac{72.000.000}{16 \times 9600} = \frac{72.000.000}{153.600} = 468,75$

2. **Parte Entera (Mantisa):**

468 = **0x1D4** en hexadecimal.

3. **Parte Fraccionaria:**

$0,75 \times 16 = 12 = \mathbf{0xC}$ en hexadecimal.

4. **Valor Final en USART1->BRR:**

$(0x1D4 \ll 4) | 0xC = \mathbf{0x1D4C}$

13.6. Registros de USART y Banderas de Error

Bandera en USART_SR	Nombre Completo	Causa / Condición de Activación
TXE	Transmit Data Register Empty	Se pone en 1 cuando el dato de <code>USART_DR</code> pasó al registro de desplazamiento. Seguro para escribir nuevo dato.
TC	Transmission Complete	Se pone en 1 cuando se completó la emisión total del bit de Stop de la última trama.
RXNE	Read Data Register Not Empty	Se pone en 1 cuando se recibió un byte completo y está listo para ser leído en <code>USART_DR</code> .
ORE	Overrun Error	Se recibió un nuevo byte en el registro de desplazamiento cuando <code>RXNE</code> aún estaba en 1 (pérdida de datos por software lento).
FE	Framing Error	No se detectó el bit de Stop en el tiempo esperado (desajuste de baud rate o ruido severo).
PE	Parity Error	El bit de paridad recibido no coincide con el cálculo del hardware.

13.7. Código de Transmisión y Recepción UART por Polling en C

```
// Transmitir un byte por USART1 (bloqueante)
void USART1_WriteChar(char c)
{
    while (!(USART1->SR & USART_SR_TXE)); // Esperar buffer libre
    USART1->DR = (uint16_t)(c & 0xFF);
}

// Recibir un byte por USART1 (bloqueante)
char USART1_ReadChar(void)
{
    while (!(USART1->SR & USART_SR_RXNE)); // Esperar dato disponible
    return (char)(USART1->DR & 0xFF);
}
```

MÓDULO 6: SISTEMAS OPERATIVOS EN TIEMPO REAL (FreeRTOS)

Capítulo 14: Paradigma Super-Loop vs RTOS y Conceptos de Tiempo Real

14.1. El Enfoque Tradicional Bare-Metal ("Super-Loop" / Big Loop)

En el desarrollo de microcontroladores sin sistema operativo, el patrón clásico consiste en un bucle infinito `while(1)` que ejecuta secuencialmente las funciones del sistema:

```
+-----+
|                                     |
|               EL PATRÓN SUPER-LOOP |
|                                     |
| int main(void) {                   |
|     Hardware_Init();               |
|     while (1) {                   |
|         Leer_Sensores(); // Si una función se demora o bloquea, |
|         Procesar_Control();// TODO el resto del sistema sufre latencia, |
|         Actualizar_Display(); // retardo acumulado (jitter) y pérdida de eventos. |
|         Enviar_Telemetria();       |
|     }                               |
| }                                   |
+-----+
```

Las 5 Reglas Obligatorias de Diseño en un Super-Loop:

1. **No ser Bloqueantes:** Ninguna función puede contener bucles de espera activa (busy-waiting) ni demoras artificiales (ej. `delay_ms()`).
2. **Ejecución Ultrarrápida (Fast Execution):** Cada función debe realizar una porción mínima de trabajo y retornar de inmediato. La latencia del sistema equivale a la sumatoria del tiempo de ejecución de todas las funciones del bucle.
3. **Divisibilidad mediante Máquinas de Estado:** Toda tarea prolongada debe dividirse en estados internos (usando variables `static enum`) para ejecutarse en pequeños pasos a través de sucesivas iteraciones del bucle principal.
4. **Reentrancia Obligatoria ante Interrupciones:** Si una función puede ser interrumpida y llamada desde una ISR, no puede compartir buffers globales sin protección atómica.
5. **Uso de Variables Globales para Paso de Estado:** Al no existir colas gestionadas por el sistema operativo, las funciones deben coordinarse mediante flags y buffers globales, incrementando el acoplamiento y el riesgo de condiciones de carrera.

Limitaciones Severas del Super-Loop:

1. Bloqueo en Cascada:

Si un sensor I2C no responde y una función se queda esperando en un bucle, se congela todo el microcontrolador.

2. Latencia Impredecible y Jitter:

El tiempo que tarda en atenderse una entrada depende de cuántas funciones y bifurcaciones condicionales se ejecuten antes en ese ciclo particular.

3. Inescalabilidad:

Agregar un nuevo módulo o sensor altera los tiempos de todas las demás tareas del bucle.

4. Desperdicio Energético:

La CPU opera al 100% de carga constante ejecutando sondeos (polling) inútiles cuando no hay eventos pendientes.

14.2. Definición de Tiempo Real: Hard vs Soft Real-Time

CONCEPTO CLAVE DE EXAMEN:

Un

Sistema de Tiempo Real (RTOS)

NO es un sistema que simplemente ejecuta instrucciones a máxima velocidad; es un sistema que **garantiza el determinismo temporal y el cumplimiento estricto de plazos temporales (deadlines)** ante eventos del mundo real.

- **Hard Real-Time (Tiempo Real Estricto):** El incumplimiento de un solo plazo temporal se considera una **falla catastrófica total del sistema**, con riesgo de daños materiales o pérdidas de vidas humanas.
Ejemplos: Despliegue de bolsas de aire (Airbags), control de frenado ABS, inyección de combustible en motores, sistemas de soporte de vida e instrumental médico crítico.
- **Soft Real-Time (Tiempo Real No Estricto):** El incumplimiento ocasional de un plazo temporal **degrada la calidad del servicio (QoS)**, pero no destruye el sistema ni pone en riesgo vidas humanas.
Ejemplos: Refresco de una pantalla LCD/OLED, streaming de audio/video, registro de logs en tarjeta SD, actualización de interfaz de usuario táctil.

14.3. Ventajas Arquitectónicas de un RTOS (FreeRTOS)

- **Multitarea Concurrente Preemptiva:** El código se divide en **Tareas (Tasks) independientes**. Cada tarea se escribe como si tuviera la CPU dedicada exclusivamente para ella en un bucle `for(;;)` simple y legible.
- **Planificador Basado en Prioridades:** La tarea de mayor prioridad que esté lista para ejecutarse (Ready) toma el control inmediato de la CPU, desalojando (preempting) a cualquier tarea de menor jerarquía.
- **Diseño Guiado por Eventos (Event-Driven):** Cuando una tarea espera un dato (de una cola, semáforo o timer), pasa al estado **Blocked (Bloqueada)** y consume exactamente **0% de CPU**.
- **Máxima Eficiencia Energética:** Cuando todas las tareas de la aplicación están bloqueadas esperando eventos, el kernel ejecuta la tarea `Idle`, la cual puede poner al procesador en modo de bajo consumo (Sleep / WFI) automáticamente.

Capítulo 15: FreeRTOS - Gestión de Tareas, Planificación y Temporización

15.1. Convenciones y Notación Húngara en FreeRTOS

FreeRTOS utiliza una rigurosa convención de nombres para tipado de variables y funciones:

- **Prefijos de Tipos:** `c` (char), `s` (short/int16), `l` (long/int32), `u` (unsigned), `x` (BaseType_t, handles, structs), `p` (puntero, ej. `pxHigherPriorityTaskWoken`), `v` (void / sin retorno).
- **Prefijos de Alcance:** `prv` (función privada / estática de archivo).
- **Tipos Fundamentales:**
 - `TickType_t`: Variable entera para conteo de ticks temporales del sistema (16 o 32 bits según `configUSE_16_BIT_TICKS`).
 - `BaseType_t`: Tipo de dato entero más eficiente de la arquitectura (en ARM Cortex-M de 32 bits equivale a `int32_t`).
- **Constantes Booleanas:** `pdTRUE` / `pdPASS` (1), `pdFALSE` / `pdFAIL` (0), `portMAX_DELAY` (espera infinita).

15.2. Anatomía de una Tarea y la Función `xTaskCreate()`

Una tarea de FreeRTOS es una función en C que **nunca debe retornar** (contiene un bucle infinito) y posee su propio Stack privado y su bloque de control **TCB (Task Control Block)**:

```
void vTaskCodigo(void *pvParameters)
{
    // Variables locales (residen en el stack propio de la tarea)
    int contador = 0;

    for (;;) {
        // Código de la tarea
        vTaskDelay(pdMS_TO_TICKS(100)); // Cede la CPU durante 100 ms
    }

    // SI por alguna razón sale del bucle, DEBE autoeliminar-se:
    vTaskDelete(NULL);
}
```


Creación de Tareas con xTaskCreate:

```
BaseType_t xTaskCreate(  
    TaskFunction_t pvTaskCode,        // Puntero a la función de la tarea  
    const char * const pcName,        // Nombre descriptivo de texto (para debug)  
    configSTACK_DEPTH_TYPE usStackDepth, // Tamaño del Stack en PALABRAS (words = 4 bytes)  
    void *pvParameters,              // Parámetro genérico pasado a la tarea  
    UBaseType_t uxPriority,           // Prioridad de la tarea (0 = mínima)  
    TaskHandle_t *pxCreatedTask       // Handle para referenciar la tarea (opcional)  
);
```

OJO - Trampa de Examen:

En FreeRTOS,

el tamaño del stack (usStackDepth) se expresa en PALABRAS (words de 32 bits = 4 bytes en ARM), NO en bytes

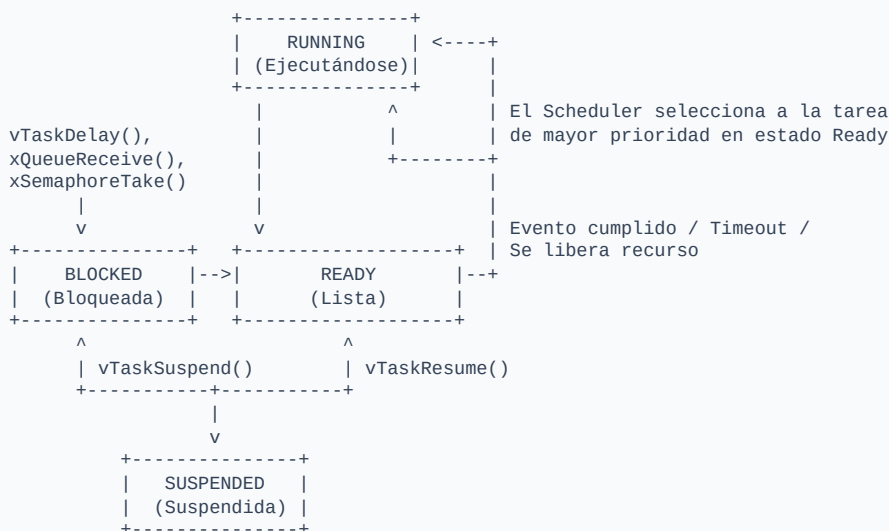
. Pasar `1000` reserva `1000 * 4 = 4000 bytes` en el Heap.

Asimismo, en FreeRTOS

el valor numérico de prioridad es DIRECTO: 0 es la prioridad MÁS BAJA

, y números mayores representan prioridades superiores (al revés que el NVIC de ARM).

15.3. Máquina de Estados de una Tarea en FreeRTOS



- **Running (En Ejecución):** La tarea posee el control de la CPU (máximo 1 tarea por núcleo en Cortex-M).
- **Ready (Lista):** La tarea está en condiciones de ejecutarse inmediatamente, esperando que el Scheduler le asigne la CPU cuando no haya tareas de mayor prioridad ejecutándose.
- **Blocked (Bloqueada):** La tarea espera por un **evento temporal** (expiración de un delay) o por un **evento de sincronización** (llegada de un dato a una cola, liberación de un semáforo/mutex). *No consume ciclos de reloj de CPU.*
- **Suspended (Suspendida):** La tarea fue puesta fuera de servicio explícitamente mediante `vTaskSuspend()`. No es evaluada por el planificador hasta que otra tarea ejecute `vTaskResume()`.

15.4. Algoritmos de Selección de Tarea del Scheduler

FreeRTOS permite seleccionar dos métodos de planificación en `FreeRTOSConfig.h`:

1. **Método Genérico en C (`configUSE_PORT_OPTIMISED_TASK_SELECTION = 0`):**
 - Escrito 100% en C ANSI estándar (máxima portabilidad a cualquier arquitectura).
 - No impone límite en el número de prioridades disponibles.
 - El Scheduler busca la tarea de mayor prioridad recorriendo linealmente las listas de tareas listos mediante bucles (complejidad $O(N)$).
2. **Método Optimizado por Puerto (`configUSE_PORT_OPTIMISED_TASK_SELECTION = 1`):**
 - Escrito en código ensamblador específico de ARM Cortex-M.

- Límite de hasta 32 niveles de prioridad (0 a 31).
- Utiliza un mapa de bits de 32 bits y la instrucción ensamblador **CLZ (Count Leading Zeros)** del núcleo ARM Cortex-M3. Encuentra la tarea de mayor prioridad en **un único ciclo de reloj constante** $O(1)$.

15.5. Fenómeno de Inanición (Starvation) por Polling Activo

Si una tarea de alta prioridad realiza una espera activa mediante un bucle de sondeo (busy-waiting como `while(flag == 0);`), la tarea **nunca abandona el estado Running**. Al ser de mayor prioridad, el Scheduler jamás le cede la CPU a tareas de menor jerarquía, produciendo la **inanición total (starvation)** de las demás tareas y de la tarea `Idle`.

15.6. Temporización: `vTaskDelay()` vs `vTaskDelayUntil()`

Característica	<code>vTaskDelay(xTicksToDelay)</code>	<code>vTaskDelayUntil(&pxPreviousWakeTime, xTimeIncrement)</code>
Tipo de Retardo	Relativo: Bloquea la tarea por una cantidad de ticks contados a partir del momento exacto en que se invoca la función.	Absoluto / Periódico: Bloquea la tarea hasta un instante temporal absoluto futuro, garantizando una frecuencia de ejecución constante.
Comportamiento Temporal	Sufre deslizamiento temporal y acumulación de error (Jitter / Drift) . El período real de repetición equivale a: $extPeriodoReal = extTiempodeEjecuciondeLaTarea + extDelay$	Determinismo estricto sin drift: El período de ejecución es exactamente constante independientemente del tiempo que tarde la tarea en ejecutarse.
Aplicación Recomendada	Temporizaciones simples, parpadeo de LEDs no críticos, timeouts.	Lazos de control en tiempo real (PID), muestreo de filtros digitales (DSP), lectura periódica estricta de sensores.

Código Comparativo de Tarea Periódica Determinista:

```
// Implementación con vTaskDelayUntil (Período exacto de 50 ms)
void vPeriodicTask(void *pvParameters)
{
    TickType_t xLastWakeTime = xTaskGetTickCount(); // Inicializar con el tiempo actual
    const TickType_t xFrequency = pdMS_TO_TICKS(50); // 50 ms

    for (;;) {
        // Bloquear hasta el próximo múltiplo absoluto de 50 ms
        vTaskDelayUntil(&xLastWakeTime, xFrequency);

        // Código de control crítico (muestreo, cálculo de lazo cerrado)
        Ejecutar_Control_PID();
    }
}
```

Capítulo 16: Tarea Idle, Idle Hook y Prioridades Dinámicas

16.1. La Tarea Ociosa (Idle Task)

Cuando se inicia el planificador mediante `vTaskStartScheduler()`, el kernel crea automáticamente una tarea interna de fondo denominada **Idle Task (prvIdleTask)** con **prioridad 0 (la más baja posible)**.

Funciones Críticas de la Tarea Idle:

- Garantizar que siempre haya al menos una tarea en estado Ready:** La CPU nunca puede quedar flotante sin instrucciones que ejecutar. Si todas las tareas de la aplicación están en estado Blocked, el Scheduler ejecuta a Idle.
- Liberar la Memoria Dinámica de Tareas Eliminadas:** Cuando una tarea se autoelimina mediante `vTaskDelete(NULL)`, no puede liberar su propio stack mientras lo está utilizando. El kernel marca la tarea y es **la tarea Idle la encargada de liberar físicamente la memoria del Stack y del TCB** de vuelta al Heap de FreeRTOS.

16.2. El Gancho de la Tarea Ociosa (Idle Hook Function)

Si en `FreeRTOSConfig.h` se habilita `configUSE_IDLE_HOOK = 1`, el kernel invocará automáticamente la función callback definida por el usuario `vApplicationIdleHook(void)` en cada iteración de la tarea Idle.

- **Usos Típicos del Idle Hook:**

- Poner al procesador en modo de ahorro de energía ejecutando la instrucción ensamblador `__WFI()` (Wait For Interrupt).
- Medir el porcentaje de tiempo ocioso de la CPU para cálculo de métricas de carga del sistema.
- Ejecutar rutinas de verificación de integridad de memoria en segundo plano.

REGLA DE ORO DEL IDLE HOOK (Pregunta Clásica de Examen):

La función `vApplicationIdleHook()`

NUNCA DEBE llamar a funciones de la API que puedan bloquear a la tarea

(como `vTaskDelay()` o lecturas de colas con timeout). Si la tarea Idle se bloquea, no habrá ninguna tarea lista para ejecutar en la CPU, provocando el colapso del sistema operativo.

16.3. Prioridades Dinámicas y Eliminación de Tareas

- **Modificación Dinámica de Prioridades:**

```
// Modificar prioridad de una tarea en tiempo de ejecución
vTaskPrioritySet(xTaskHandle, uxNewPriority);

// Consultar la prioridad actual
UBaseType_t prio = uxTaskPriorityGet(xTaskHandle);
```

Comportamiento: Si una tarea eleva su prioridad por encima de la tarea actualmente en ejecución, el Scheduler produce un **desalojo inmediato** (Context Switch en ese mismo instante). Requiere `INCLUDE_vTaskPrioritySet = 1`.

- **Eliminación de Tareas:** `vTaskDelete(xTaskHandle);` retira a la tarea de todas las listas del Scheduler y la marca para destrucción.

Capítulo 17: Colas de Mensajes (Queues)

17.1. Inconvenientes de las Variables Globales en Multitarea

El método tradicional de comunicar tareas mediante variables globales presenta graves riesgos:

- **Condiciones de Carrera (Race Conditions):** Una interrupción o un cambio de contexto en medio de una lectura/escritura no atómica de una variable multibyte corrompe el dato.
- **Polling Ineficiente:** Requiere que la tarea receptora sondee continuamente la variable consumiendo el 100% de la CPU.
- **Ausencia de Buffering (Pérdida de Datos):** Si el emisor produce varios datos antes de que el receptor los procese, los valores intermedios se sobrescriben.
- **Fuerte Acoplamiento:** El emisor debe conocer el momento exacto en que el receptor está listo.

17.2. Características y Almacenamiento: "Copia por Valor"

Una **Cola (Queue)** en FreeRTOS es un buffer FIFO (First-In, First-Out) **thread-safe** gestionado por el kernel, con capacidad para un número finito de elementos de tamaño fijo.

Mecanismo de Copia por Valor (Por Valor vs Por Referencia):

FreeRTOS implementa internamente el almacenamiento

Por Copia de Datos (byte a byte)

- **Ventajas Fundamentales:**

1. **Desacoplamiento Total:**

El emisor y el receptor no comparten punteros ni memoria.

2. **Reutilización Inmediata:**

El emisor puede modificar, sobrescribir o liberar la variable original inmediatamente después de llamar a `xQueueSend()`.

3. **Seguridad con Variables Locales del Stack:**

Es 100% seguro pasar a la cola una variable local asignada en la pila, ya que el kernel copia su contenido antes de que la función retorne.

17.3. Bloqueo en Lectura y Escritura de Colas

- **Bloqueo en Lectura (Cola Vacía):** Cuando una tarea intenta leer de una cola sin elementos (`xQueueReceive()`), el kernel la coloca en estado **Blocked** por un tiempo máximo `xTicksToWait` (0% CPU). En el instante en que otra tarea o ISR escribe un dato, el kernel **desbloquea inmediatamente a la tarea lectora**. Si hay varias tareas esperando, se desbloquea la de *mayor prioridad* (o la que más tiempo lleva esperando a igual prioridad).
- **Bloqueo en Escritura (Cola Llena):** Si una tarea intenta enviar a una cola completa (`xQueueSendToBack()`), se bloquea hasta que un lector extraiga un elemento y libere espacio.

17.4. API Fundamental de Colas

Función de la API	Parámetros Principales	Retorno	Descripción Funcional
<code>xQueueCreate()</code>	<code>uxQueueLength</code> (cantidad elementos), <code>uxItemSize</code> (tamaño de cada elemento en bytes)	<code>QueueHandle_t</code> (o <code>NULL</code> si no hay RAM)	Crea dinámicamente la cola en el Heap de FreeRTOS.
<code>xQueueSendToBack()</code> / <code>xQueueSend()</code>	<code>xQueue</code> , <code>pvItemToQueue</code> (puntero al dato a copiar), <code>xTicksToWait</code>	<code>pdPASS</code> / <code>errQUEUE_FULL</code>	Inserta el elemento al final de la cola (disciplina FIFO).
<code>xQueueSendToFront()</code>	<code>xQueue</code> , <code>pvItemToQueue</code> , <code>xTicksToWait</code>	<code>pdPASS</code> / <code>errQUEUE_FULL</code>	Inserta el elemento al inicio de la cola (mensajes urgentes LIFO).
<code>xQueueReceive()</code>	<code>xQueue</code> , <code>pvBuffer</code> (donde copiar el dato leído), <code>xTicksToWait</code>	<code>pdPASS</code> / <code>errQUEUE_EMPTY</code>	Extrae y elimina el elemento al frente de la cola.
<code>uxQueueMessagesWaiting()</code>	<code>xQueue</code>	<code>UBaseType_t</code>	Retorna la cantidad de elementos disponibles actualmente en la cola.

17.5. Patrones de Diseño Avanzados con Colas

1. Múltiples Fuentes a una Sola Cola Receptora (Estructuras con ID)

Cuando un controlador centralizado recibe datos de distintas fuentes (ej. tarea HMI, tarea Bus CAN, tarea Sensores), se define una estructura que incluye un enumerador de origen:

```
typedef enum { eSensorTemperatura, eSensorPresion, eComandoHMI } DataSource_t;

typedef struct {
    DataSource_t eOrigen;
    int32_t lValor;
} Mensaje_t;

// Creación de la cola de estructuras
QueueHandle_t xColaEventos = xQueueCreate(10, sizeof(Mensaje_t));
```

2. Envío de Datos Grandes o Variables mediante Punteros (Por Referencia)

Copiar buffers masivos (ej. paquetes Ethernet de 1500 bytes o imágenes) en la cola consume demasiada RAM y tiempo de CPU. En este caso, **la cola se crea para almacenar únicamente el tamaño de un puntero (sizeof(void *))**:

```
// Creación de cola de punteros (10 punteros)
QueueHandle_t xColaPunteros = xQueueCreate(10, sizeof(char *));

// Tarea Emisora:
char *pcBuffer = (char *)pvPortMalloc(128); // 1. Asigna memoria dinámica
snprintf(pcBuffer, 128, "Mensaje de telemetria"); // 2. Rellena datos
xQueueSend(xColaPunteros, &pcBuffer, portMAX_DELAY); // 3. Envía la DIRECCIÓN del puntero

// Tarea Receptora:
char *pcMensajeRecibido;
xQueueReceive(xColaPunteros, &pcMensajeRecibido, portMAX_DELAY); // 1. Recibe el puntero
vProcesar(pcMensajeRecibido); // 2. Procesa los datos
vPortFree(pcMensajeRecibido); // 3. LIBERA la memoria
```

Reglas Estrictas al Enviar Punteros en Colas:

1. Propiedad Única y Clara:

Una vez que el emisor coloca el puntero en la cola,
no debe volver a leer ni modificar esa memoria jamás
. La propiedad se transfiere exclusivamente al receptor.

2. Memoria Válida:

NUNCA enviar punteros a variables locales del stack de una función que pueda finalizar. Utilizar buffers dinámicos del Heap o buffers estáticos globales.

3. Liberación de Memoria:

Exactamente
una sola tarea
debe ser la responsable de liberar el buffer con `vPortFree()` tras procesarlo.

Capítulo 18: Gestión de Interrupciones y Procesamiento Diferido

18.1. Prioridad de Hardware (NVIC) vs Prioridad de Software (FreeRTOS)

PRINCIPIO ABSOLUTO DE TIEMPO REAL: EL HARDWARE TIENE PRECEDENCIA TOTAL

1. La prioridad de una tarea de FreeRTOS es un concepto de **software**

gestionado por el Scheduler.

2. La prioridad de una interrupción en el NVIC es un concepto de **hardware**

gestionado por los comparadores eléctricos de la CPU.

3.

La interrupción de menor prioridad del NVIC siempre interrumpirá inmediatamente a la tarea de FreeRTOS de mayor prioridad.

Ninguna tarea de software puede jamás desalojar o interrumpir a una ISR de hardware.

18.2. El API Seguro para Interrupciones (...FromISR)

Dentro de una Rutina de Servicio de Interrupción (ISR) rige una restricción fundamental: **UNA ISR NUNCA PUEDE BLOQUEARSE** (no puede llamar a `vTaskDelay` ni esperar recursos con timeout). Por lo tanto, FreeRTOS proporciona versiones especializadas y seguras para ISR que terminan en `...FromISR()`.

PROHIBIDO:

Llamar a funciones del API estándar (como `xQueueSend` o `xSemaphoreTake`) dentro de una ISR. Provoca la corrupción del sistema operativo. Solo deben invocarse funciones `...FromISR()`.

18.3. El Parámetro `pxHigherPriorityTaskWoken` y Cambio de Contexto Forzado

Cuando una función como `xSemaphoreGiveFromISR()` o `xQueueSendFromISR()` desbloquea a una tarea que estaba esperando ese evento, si dicha tarea tiene una **prioridad de software mayor que la tarea que estaba corriendo antes de la interrupción**, la función de FreeRTOS coloca el puntero `*pxHigherPriorityTaskWoken = pdTRUE`.

Al finalizar la ISR, el desarrollador pasa esta variable a la macro `portYIELD_FROM_ISR()` (o `portEND_SWITCHING_ISR()`):

```
void EXTI0_IRQHandler(void)
{
    BaseType_t xHigherPriorityTaskWoken = pdFALSE; // 1. SIEMPRE inicializar en pdFALSE

    // 2. Atender hardware y limpiar bandera del periférico
    EXTI->PR = EXTI_PR_PR0;

    // 3. Notificar a la tarea mediante semáforo o cola
    xSemaphoreGiveFromISR(xSemBoton, &xHigherPriorityTaskWoken);

    // 4. Si se despertó una tarea de mayor prioridad, forzar cambio de contexto inmediato
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
}
```

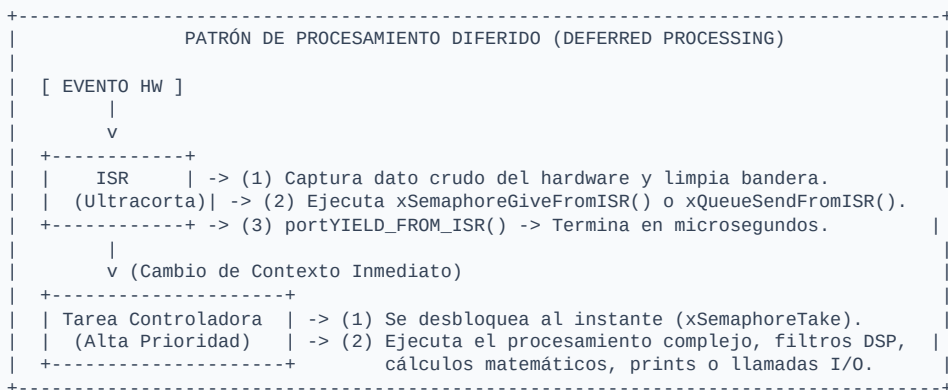
Resultado: En lugar de retornar a la tarea de baja prioridad que fue interrumpida, el procesador **conmuta en ese mismo instante a la tarea de alta prioridad**, ejecutando la respuesta al evento sin esperar al siguiente tick del reloj del sistema.

18.4. Procesamiento Diferido de Interrupciones (Deferred Interrupt Processing)

Una regla de oro en sistemas embebidos de tiempo real es: **Mantener las Rutinas de Servicio de Interrupción (ISR) lo más breves y rápidas posible.**

Razones para no sobrecargar una ISR:

1. Mientras corre una ISR, se bloquea la atención de otras interrupciones de igual o menor prioridad.
2. Aumenta el jitter e impredecibilidad temporal de todas las tareas críticas del sistema.
3. Dentro de una ISR no se pueden realizar operaciones lentas o no deterministas (como formateo de texto, escrituras lentas por UART, o asignación de memoria dinámica `malloc`).



18.5. Sincronización con Semáforos Binarios

Un **Semáforo Binario** en FreeRTOS es el mecanismo más simple y eficiente para sincronizar una tarea con una interrupción. Puede pensarse conceptualmente como una **cola de longitud 1**:

- **Creación:** Se crea con `xSemaphoreCreateBinary()`. Un semáforo binario **se crea en estado "vacío"** (no disponible para ser tomado).
- **La Tarea (Consumidor):** Ejecuta `xSemaphoreTake(xSem, portMAX_DELAY)` y se **bloquea consumiendo 0% de CPU** a la espera del evento.

- **La ISR (Productor):** Cuando ocurre el pulso del sensor, ejecuta `xSemaphoreGiveFromISR(xSem, &xHigherPriorityTaskWoken)`, liberando el semáforo y despertando a la tarea inmediatamente.

Código Completo de Sincronización ISR-Tarea:

```
SemaphoreHandle_t xBinarySemaphore;

// Tarea de Procesamiento Diferido (Alta Prioridad: Prioridad 3)
void vHandlerTask(void *pvParameters)
{
    for (;;) {
        // Bloquear indefinidamente hasta que la ISR entregue el semáforo
        if (xSemaphoreTake(xBinarySemaphore, portMAX_DELAY) == pdPASS) {
            // Procesamiento pesado del evento del sensor
            vProcesarDatosComplejos();
        }
    }
}

// Rutina de Interrupción del Periférico
void Sensor_IRQHandler(void)
{
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    // Limpiar bandera de interrupción en el hardware
    SENSOR->SR &= ~SENSOR_SR_IF;

    // Dar el semáforo para desbloquear la tarea handler
    xSemaphoreGiveFromISR(xBinarySemaphore, &xHigherPriorityTaskWoken);

    // Forzar cambio de contexto inmediato a vHandlerTask
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
}

// Inicialización en main()
int main(void)
{
    xBinarySemaphore = xSemaphoreCreateBinary();

    if (xBinarySemaphore != NULL) {
        xTaskCreate(vHandlerTask, "Handler", 1000, NULL, 3, NULL);
        vTaskStartScheduler();
    }
    while (1);
}
```

MÓDULO 7: BANCO DE PREGUNTAS TÍPICAS DE EXAMEN Y RESPUESTAS

MODELO

Este banco recopila 30 preguntas conceptuales, numéricas y prácticas de alta probabilidad de examen para el 1er Parcial Teórico de Técnicas Digitales II, con sus respuestas modelo desarrolladas con máximo rigor técnico.

1. ¿Qué establece la Ley de Moore y cuáles fueron sus consecuencias técnicas y económicas directas?

Respuesta: Formulada por Gordon Moore en 1965, establece que la cantidad de transistores integrados en un chip de silicio se duplica aproximadamente cada 18 a 24 meses. Sus consecuencias directas son: (1) Reducción drástica del costo por función lógica/transistor, posibilitando la masificación de sistemas embebidos; (2) Aumento de la frecuencia de conmutación de los circuitos al reducirse las dimensiones parásitas; (3) Integración de sistemas completos en un solo chip (SoC).

2. ¿Cuáles son las cuatro restricciones fundamentales de diseño que diferencian a los Sistemas Embebidos de las PCs y Servidores?

Respuesta: (1) **Costo extremo:** Al fabricarse en grandes volúmenes, cada centavo de ahorro en componentes es decisivo; (2) **Consumo de energía:** Muchos operan con baterías y requieren disipación térmica exclusivamente pasiva; (3) **Alta Confiabilidad / Disponibilidad:** Un fallo puede provocar pérdidas humanas o daños catastróficos (ej. aviónica, automotriz ABS, instrumental médico); (4) **Procesamiento en Tiempo Real Determinista:** Deben responder a eventos físicos en plazos temporales estrictos y acotados.

3. Explique la diferencia entre Kilobyte (KB) y Kibibyte (KiB) según la norma IEC 80000-13 y por qué es crítico en arquitectura de computadores.

Respuesta: El Kilobyte (KB) es una unidad decimal del SI equivalente a $10^3 = 1.000$ bytes, mientras que el Kibibyte (KiB) es una unidad binaria estándar equivalente a $2^{10} = 1.024$ bytes. En arquitectura de computadores y direccionamiento de microcontroladores (ej. Flash y SRAM), los buses operan en potencias de 2 (KiB). Confundir estas unidades introduce discrepancias de cálculo que crecen desde un 2,4% en KiB hasta un 20,9% en YiB.

4. Describa brevemente 4 de las 8 Grandes Ideas de la Arquitectura de Computadores propuestas por Patterson y Hennessy.

Respuesta: (1) **Diseño para la Ley de Moore:** Proyectar los desarrollos considerando la densidad tecnológica que existirá cuando el chip termine su fase de diseño; (2) **Abstracción:** Ocultar la complejidad de bajo nivel mediante capas jerárquicas; (3) **Hacer el caso común más rápido:** Optimizar los caminos de ejecución más frecuentes; (4) **Jerarquía de Memorias:** Emplear cachés rápidas y pequeñas cerca de la CPU y memorias más lentas y densas en niveles inferiores para explotar la localidad de datos.

5. ¿Qué es una ISA (Instruction Set Architecture) y por qué garantiza la independencia de implementación?

Respuesta: La ISA es la interfaz abstracta que define las instrucciones, registros, modos de direccionamiento y modelo de memoria visibles por el programador y el compilador. Permite la independencia de implementación porque dos procesadores con circuitería interna y litografía completamente diferentes (ej. procesadores Intel y AMD) pueden ejecutar exactamente el mismo binario siempre que ambos cumplan la misma especificación de ISA (x86-64).

6. Escriba la Ecuación del Rendimiento de CPU y explique cómo impacta el compilador y la ISA en cada una de sus variables.

Respuesta: $\text{Tiempo CPU} = \frac{I \times \text{CPI}}{f_{\text{clock}}}$. El **compilador** determina la cantidad total de instrucciones ejecutadas (I) y optimiza la combinación de instrucciones afectando el CPI promedio. La **ISA** define la complejidad de las instrucciones disponibles (lo que afecta I y el CPI) e impone restricciones al camino de datos que condicionan la frecuencia máxima (f_{clock}).

7. ¿Qué es el "Muro de Potencia" (Power Wall), qué ecuación física lo describe y qué cambio de paradigma provocó en 2006?

Respuesta: El Muro de Potencia fue el límite alcanzado cerca de 2004 donde no fue posible seguir aumentando la frecuencia de los procesadores mononúcleo debido a la imposibilidad de reducir más el voltaje sin disparar las corrientes de fuga (leakage) y a los límites térmicos de disipación de calor ($P \approx \frac{1}{2} CV^2 f$). Provocó el cambio de paradigma de 2006 hacia **procesadores multinúcleo (Multicore)**, obligando al desarrollo de software concurrente y paralelizable.

8. Compare la filosofía RISC vs CISC en relación al conjunto de instrucciones, formato y acceso a memoria.

Respuesta: CISC posee cientos de instrucciones complejas de tamaño variable que pueden operar directamente sobre datos en memoria. RISC posee un conjunto reducido de instrucciones simples de tamaño fijo (32/16 bits) con decodificación en 1 ciclo y adopta una **arquitectura Load/Store**: las operaciones aritméticas/lógicas se realizan exclusivamente entre registros del procesador; sólo las instrucciones `LDR` y `STR` acceden a la memoria.

9. ¿Cuál es la diferencia entre Arquitectura Mapeada en Memoria (Memory-Mapped I/O) y Mapeada en Puertos (Port-Mapped I/O)? ¿Cuál usa ARM y por qué?

Respuesta: En Port-Mapped I/O la memoria y los periféricos tienen espacios de direcciones aislados y requieren instrucciones especiales (ej. `IN` / `OUT` en x86). En **Memory-Mapped I/O** los periféricos y la memoria comparten un único espacio unificado de 4 GiB y se accede a ellos con las mismas instrucciones de carga y almacenamiento (`LDR` / `STR`). ARM usa Memory-Mapped I/O porque ofrece ortogonalidad total en el juego de instrucciones y permite manipular registros de periféricos directamente en lenguaje C mediante punteros y estructuras estándar.

10. ¿Por qué los registros de los periféricos deben declararse obligatoriamente con el calificador 'volatile' en C?

Respuesta: Porque un registro de periférico puede cambiar su contenido en cualquier momento por causas externas al flujo del software (llegada de un dato UART, timer, etc.). El calificador `volatile` prohíbe al optimizador del compilador guardar el valor en un registro de CPU para lecturas repetidas y evita la eliminación de bucles de espera de hardware (como `while(!(USART1->SR & RXNE));`).

11. ¿Por qué el NVIC y el SysTick están ubicados en el Private Peripheral Bus (PPB) a partir de la dirección 0xE0000000?

Respuesta: Por tres motivos clave: (1) **Determinismo y Latencia Mínima:** El PPB es un bus privado de 32 bits que no compite por ancho de banda con DMA ni periféricos de buses de sistema (AHB/APB); (2) **Seguridad:** Solo es accesible en Modo Privilegiado; (3) **Estandarización y Portabilidad:** Están en las mismas direcciones exactas en cualquier chip ARM Cortex-M de cualquier fabricante, permitiendo código de SO 100% portable.

12. Explique por qué el Ensamblador debe realizar dos pasadas sobre el archivo fuente.

Respuesta: Para resolver las **referencias hacia adelante (forward references)**, donde una instrucción hace un salto a una etiqueta que aún no ha sido declarada. En la **Primera Pasada** construye la Tabla de Símbolos calculando las direcciones relativas de todas las etiquetas. En la **Segunda Pasada** traduce las instrucciones a binario reemplazando los símbolos por sus offsets exactos y genera la tabla de relocalización.

13. Explique la diferencia entre las secciones .text, .data y .bss de un archivo objeto (.o). ¿Cuál de ellas no ocupa espacio en el binario Flash?

Respuesta: `.text` contiene el código máquina ejecutable y constantes de sólo lectura (ROM/Flash). `.data` contiene variables globales inicializadas con valores distintos de cero (LMA en Flash, VMA en RAM). `.bss` contiene variables globales no inicializadas o inicializadas en cero; **la sección .bss no ocupa espacio en el binario Flash**, solo indica su tamaño para que el Startup la inicialice a cero en RAM.

14. Explique la dualidad LMA vs VMA para la sección .data en un sistema embebido y cómo se implementa en el Linker Script.

Respuesta: Las variables inicializadas requieren que su valor inicial persista ante cortes de energía (deben residir en Flash - **LMA: Load Memory Address**), pero deben ser modificables durante la ejecución (deben residir en RAM - **VMA: Virtual/Execution Memory Address**). En el Linker Script se implementa con la sintaxis: `.data : { _sdata = .; *(<data*>); _edata = .; } > RAM AT > FLASH`, exportando la dirección de Flash mediante `_sidata = LOADADDR(.data);`.

15. Detalle las 5 responsabilidades secuenciales que ejecuta el archivo de arranque (Startup / Reset_Handler) antes de saltar a main().

Respuesta: (1) Instanciar la Tabla de Vectores definiendo el SP inicial y el puntero a `Reset_Handler`; (2) Copiar la sección `.data` desde Flash (LMA `_sidata`) hacia RAM (VMA `_sdata` a `_edata`); (3) Limpiar a cero la sección `.bss` en RAM (entre `_sbss` y `_ebss`); (4) Ejecutar rutinas de inicialización de hardware de reloj (`SystemInit()`); (5) Transferir el control a `main()` mediante `bl main`.

16. ¿Qué diferencia existe entre Pre-emption Priority (Prioridad de Grupo) y Sub-prioridad en el NVIC de ARM Cortex-M?

Respuesta: La **Pre-emption Priority (Grupo)** define si una nueva interrupción puede interrumpir y desalojar (anidarse) a una ISR que ya se está ejecutando (solo desaloja si su número de grupo es menor). La **Sub-prioridad** solo actúa como criterio de desempate cuando dos interrupciones del mismo grupo se solicitan *simultáneamente*; una menor subprioridad **NO desaloja** a otra del mismo grupo que ya esté en ejecución.

17. ¿Qué registros apila automáticamente el hardware del Cortex-M3 al entrar a una excepción y por qué se eligieron esos registros?

Respuesta: Apila 8 registros en el Stack activo (32 bytes): **R0, R1, R2, R3, R12, LR, PC y xPSR**. Se eligieron porque según el estándar AAPCS de ARM, estos son los registros que una función C puede sobrescribir ("caller-saved"). Al apilarlos por hardware en solo 12 ciclos, cualquier función C estándar puede actuar directamente como ISR sin requerir prólogos o epílogos en ensamblador.

18. ¿Qué es el valor EXC_RETURN, dónde se carga al entrar a una interrupción y qué sucede cuando la CPU intenta saltar a él al retornar?

Respuesta: Es un valor mágico con patrón `0xFFFFFFFx` que el hardware carga en el registro **LR** al entrar a la excepción, codificando el modo previo y la pila usada. Cuando la ISR finaliza con `bx lr`, la CPU detecta el patrón `0xFFFFFFFx`, activa la secuencia de retorno por hardware, desapila automáticamente los 8 registros, restaura el modo Thread anterior y limpia la bandera de interrupción activa en el NVIC.

19. Explique la diferencia de función entre los registros especiales PRIMASK y BASEPRI.

Respuesta: **PRIMASK** es un registro de 1 bit que al activarse (`CPSID 1`) eleva la prioridad de la CPU a 0, bloqueando *todas las interrupciones programables* (solo permite NMI, Reset y HardFault). **BASEPRI** es un registro de 8 bits que permite un enmascaramiento selectivo por umbral: bloquea solo las interrupciones con prioridad numéricamente $\geq N$, permitiendo que las interrupciones críticas con prioridad superior sigan funcionando.

20. Describa las 4 capas jerárquicas para activar cualquier periférico en un microcontrolador STM32.

Respuesta: (1) **Capa de Infraestructura (RCC):** Habilitar la señal de reloj en el bus periférico correspondiente (APB1 o APB2); (2) **Capa de Interfaz Física (GPIO/APIO):** Configurar el modo de los pines (Input, Output, Función Alternativa); (3) **Capa de Protocolo:** Configurar baud rate, polaridad, paridad y registros de control específicos; (4) **Capa Operativa:** Activar el bit de habilitación funcional del periférico (ej. `SPE`, `UE`).

21. Explique el significado de los bits CPOL y CPHA en el protocolo SPI y deduzca en qué flanco se leen los datos en Modo 0 y Modo 3.

Respuesta: **CPOL** define el nivel de reposo de SCLK (0 = Bajo, 1 = Alto). **CPHA** define la fase de muestreo (0 = Primer flanco, 1 = Segundo flanco). En **Modo 0 (CPOL=0, CPHA=0)** el reloj reposa en bajo y el dato se lee en el **primer flanco (ascendente)**. En **Modo 3 (CPOL=1, CPHA=1)** el reloj reposa en alto y el dato se lee en el **segundo flanco (ascendente)**.

22. ¿Por qué las líneas SDA y SCL del bus I2C operan en Drenador Abierto (Open-Drain) con resistencias Pull-Up y qué ventaja otorga en arbitraje?

Respuesta: Operan en drenador abierto porque ningún nodo fuerza activamente nivel alto; solo pueden drenar a 0V o quedar flotantes (Hi-Z), asegurando el nivel alto pasivamente con pull-ups. Esto genera una conexión **Wired-AND** que permite el **arbitraje multimaestro sin pérdida de datos**: si dos maestros transmiten a la vez, el que envía un '1' pero lee un '0' detecta la discrepancia, pierde el arbitraje y cede el bus inmediatamente sin corromper la trama del ganador.

23. Defina las condiciones de START y STOP en el protocolo I2C y explique la regla de validez de datos.

Respuesta: **START:** Transición de ALTO a BAJO en SDA con SCL en ALTO. **STOP:** Transición de BAJO a ALTO en SDA con SCL en ALTO. **Regla de Validez:** Durante la transmisión de bits de datos, la línea SDA sólo puede cambiar de estado cuando SCL está en BAJO; el dato debe permanecer estable durante todo el semiperíodo de SCL en ALTO.

24. En una comunicación UART 9600 8N1, ¿cuántos caracteres útiles por segundo se transfieren y qué representa el bit de START?

Respuesta: La trama 8N1 contiene 10 bits por carácter (1 Start + 8 Datos + 0 Paridad + 1 Stop). La tasa de transferencia útil es $\frac{9600}{10} = 960$ bytes/segundo. El bit de START es un nivel 0 lógico (flanco de bajada) que rompe el reposo de la línea y sincroniza los temporizadores del receptor para muestrear en el centro de cada bit.

25. Calcule el valor a cargar en el registro USART_BRR para una velocidad de 115200 baudios con USART1 en APB2 a 72 MHz.

Respuesta: (1) $USARTDIV = \frac{72.000.000}{16 \times 115200} = \frac{72.000.000}{1.843.200} = 39,0625$; (2) Mantisa: $39 = 0x27$; (3) Fracción: $0,0625 \times 16 = 1 = 0x1$; (4) Valor a escribir en `USART1->BRR`: $(0x27 \ll 4) | 0x1 = 0x271$.

26. ¿Cuál es la diferencia conceptual entre un sistema Hard Real-Time y Soft Real-Time? Dé un ejemplo de cada uno.

Respuesta: En **Hard Real-Time** el no cumplimiento de un plazo temporal (deadline) representa un fallo total y catastrófico con peligro de daños o vidas (ej. disparo de Airbag en un choque en < 20 ms). En **Soft Real-Time** el incumplimiento de un plazo solo degrada la calidad del servicio pero el sistema continúa funcionando de forma segura (ej. caída de frames en la actualización de una pantalla LCD).

27. Compare vTaskDelay() vs vTaskDelayUntil(). ¿Por qué vTaskDelay() produce deriva temporal (jitter/drift) en tareas periódicas?

Respuesta: `vTaskDelay()` introduce un retardo *relativo* a partir del momento en que se ejecuta la función, por lo que el período real es Tiempo Ejecucion + Delay, acumulando un corrimiento temporal inevitable. `vTaskDelayUntil()` calcula un retardo *absoluto* en base al tiempo del ciclo anterior, garantizando una frecuencia de ejecución periódica fija y determinista sin deriva temporal.

28. ¿Cuáles son las dos responsabilidades de la tarea Idle en FreeRTOS y cuál es la regla de oro de la función Idle Hook?

Respuesta: Responsabilidades de Idle: (1) Garantizar que siempre haya al menos una tarea en estado Ready para ejecutar en la CPU; (2) Liberar la memoria del Stack y TCB de las tareas eliminadas con `vTaskDelete()`. **Regla de oro del Idle Hook:** *ESTÁ TERMINANTEMENTE PROHIBIDO que la función `vApplicationIdleHook()` se bloquee o suspenda*, ya que si Idle se bloquea el kernel colapsa al no haber tareas listas para correr.

29. ¿Por qué FreeRTOS implementa almacenamiento "Por Copia de Valor" en sus colas y qué precauciones deben tomarse al enviar punteros?

Respuesta: Implementa copia de valor (byte a byte) para desacoplar totalmente al emisor del receptor y permitir el pasaje seguro de variables locales del stack. Al enviar punteros (para datos grandes), debe garantizarse: (1) Transferencia estricta de propiedad: el emisor no debe acceder al buffer tras enviarlo; (2) No apuntar a memoria local de stack; (3) Solo una tarea (el receptor) debe liberar la memoria con `vPortFree()`.

30. Explique el patrón de "Procesamiento Diferido de Interrupciones" en FreeRTOS utilizando semáforos binarios y `pxHigherPriorityTaskWoken`.

Respuesta: Consiste en dividir la atención del evento en dos etapas: (1) La ISR de hardware es ultracorta: atiende el periférico, limpia la bandera, desbloquea una tarea de alta prioridad con `xSemaphoreGiveFromISR(&pxHigherPriorityTaskWoken)` y solicita el cambio de contexto con `portYIELD_FROM_ISR(pxHigherPriorityTaskWoken)`; (2) Al salir la ISR, la CPU conmuta inmediatamente a la **Tarea de Procesamiento Diferido**, la cual toma el semáforo con `xSemaphoreTake()` y realiza el procesamiento complejo (filtros, cálculos, prints) en contexto de tarea, sin bloquear otras interrupciones del hardware.